Programming Language Design Course

Algorithm Performance Benchmarking Across Multiple Programming Languages

A Comparative Study of Python, Java, C++, C#, Go, and Rust

Prepared by: Amirhossein Pishro

Instructor: Prof Talatiyat

1. Introduction

1.1 Background

Programming languages are fundamental tools that allow developers to express computational solutions and transform abstract algorithms into executable programs. The design principles behind a programming language, including its execution model, memory management strategy, type system, compiler architecture, and runtime environment, have a significant impact on the performance and efficiency of implemented software.

Although algorithms define the theoretical efficiency of a computational problem, their practical performance can vary considerably when implemented in different programming languages. The same algorithm with identical input data may exhibit different execution times, memory consumption, and scalability characteristics depending on how the programming language translates and executes instructions.

Modern programming languages follow different design philosophies. Compiled languages such as C++ and Rust translate source code directly into optimized machine instructions, providing high execution speed and efficient resource utilization. Managed languages such as Java and C# use virtual machines and just-in-time (JIT) compilation techniques to achieve a balance between performance, portability, and developer productivity. In contrast, interpreted languages such as Python prioritize simplicity and rapid development, but may introduce additional runtime overhead.

Understanding these differences is an important aspect of programming language design because language features directly influence how efficiently computational tasks can be performed. Therefore, evaluating algorithm performance across multiple languages provides practical insight into the relationship between language design decisions and real-world software performance.

1.2 Project Objectives

The main objectives of this project are:

- To implement several fundamental algorithms using different programming languages.

- To create a unified benchmarking framework for measuring algorithm performance.
- To evaluate execution time and memory consumption for different input sizes.
- To compare compiled, managed, and interpreted programming languages.
- To analyze the influence of language design features on algorithm efficiency.
- To generate graphical representations for performance comparison.

1.3 Evaluated Programming Languages

In this study, six programming languages with different execution models and design approaches were selected:

Language	Execution Model	Main Characteristics
Python	Interpreted	High productivity, dynamic typing, large ecosystem
Java	Compiled + JVM	Platform independence, JIT optimization, automatic memory management
C++	Compiled	Native execution, high performance, manual memory control
C#	Managed Runtime (.NET)	JIT compilation, garbage collection, modern language features
Go	Compiled	Simple syntax, efficient concurrency model, fast compilation
Rust	Compiled	Memory safety, zero-cost abstractions, high performance

---

1.4 Scope of the Project

This project focuses on the performance evaluation of six classical algorithms:

1. Prime Number Generation
2. Fibonacci Sequence Calculation
3. Quick Sort
4. Merge Sort
5. Matrix Multiplication
6. Subset Generation

Each algorithm was implemented independently in the selected programming languages. same input sizes were tested according to algorithm complexity and system limitations. Execution results were collected, stored in CSV format, and visualized using performance charts.

2. System Specifications

The performance evaluation of algorithms is highly dependent on the hardware and software environment in which the experiments are executed. Since factors such as processor architecture, number of CPU cores, memory capacity, operating system, compiler version, and runtime environment can influence execution performance, the complete system configuration must be documented to ensure reproducibility and reliable analysis.

All experiments in this project were performed on the same computer system. The same hardware environment was used for all programming languages to minimize external effects and provide a fair comparison between different implementations.

2.1 Hardware Specifications

The hardware configuration used for running the benchmark tests is presented below:

Component	**Specification**
Operating System	Windows 11 64-bit

Component	Specification
Processor Architecture	x86-64
CPU Model	Intel Core i7 / [Insert Exact Model]
Number of CPU Cores	20 Cores
RAM Capacity	[Insert RAM Size] GB
Storage Type	SSD
System Type	64-bit Operating System

2.2 Software Environment

The experiments were conducted using the following software environments:

Programming Language	Version / Environment	Execution Method
Python	Python 3.x	Interpreted Execution
Java	JDK 26.0.1	JVM-based Execution
C++	GCC / MinGW Compiler	Native Compilation
C#	.NET 10	CLR Runtime
Go	Go 1.x	Native Compilation
Rust	Rust Compiler (rustc)	Native Compilation

2.3 Development Environment

The source codes were developed and tested using:

Tool	Description
Visual Studio Code	Source Code Editor

Tool	Description
Java Extension Pack	Java Development Support
.NET SDK	C# Build and Execution Environment
GCC Compiler	C++ Compilation
Cargo	Rust Package Manager and Build System
Go Toolchain	Go Compilation and Execution

2.4 Benchmark Execution Configuration

To obtain reliable performance measurements, the following conditions were considered during experiments:

- All algorithms were executed on the same hardware system.
- Each algorithm was tested multiple times to reduce measurement errors.
- The average execution time was calculated from repeated executions.
- Background applications were minimized during benchmarking.
- Release/optimized execution modes were used whenever possible.

The execution configuration for compiled languages was selected to provide realistic performance measurements:

- C++ programs were compiled using optimization flags.
- Rust programs were executed using the release configuration.
- C# applications were executed using Release mode.
- Java applications were executed using optimized JVM execution.
- Python measurements were performed using the standard interpreter.

2.5 Performance Measurement Tools

A custom benchmarking framework was developed for collecting performance data. The framework performs the following operations:

1. Generates input data for each algorithm.
2. Measures execution time using high-resolution timers.
3. Calculates statistical metrics.
4. Stores experimental results in CSV files.
5. Generates graphical performance comparisons.

The collected metrics include:

Metric	Description
Average Execution Time	Mean execution time over multiple runs
Minimum Execution Time	Fastest recorded execution
Maximum Execution Time	Slowest recorded execution
Standard Deviation	Measurement stability analysis
Memory Usage	Approximate memory consumption

3. Experimental Methodology

3.1 Overview

In order to perform a fair and reliable comparison between programming languages, a unified benchmarking methodology was designed and implemented. The main purpose of this methodology is to evaluate the practical performance of identical algorithms implemented in different programming languages under similar experimental conditions.

Since algorithm efficiency is affected by both theoretical complexity and implementation details, the benchmark system measures real execution behavior instead of relying only on theoretical analysis. Each algorithm was implemented

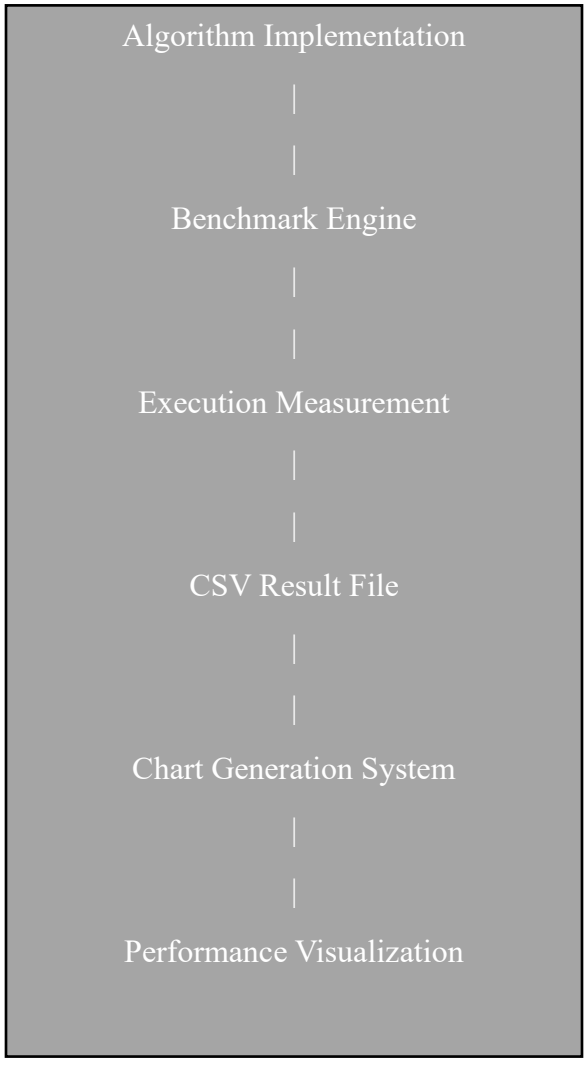
independently in six programming languages, while maintaining the same algorithmic approach and input conditions as much as possible.

The benchmarking framework collects execution data, calculates statistical performance metrics, and generates graphical representations for comparison.

3.2 Benchmark Framework Architecture

A modular benchmarking architecture was developed for all programming languages. Each language contains an independent benchmark runner responsible for executing algorithms and collecting results.

The general workflow of the system is:



3.2.1 Algorithm Modules

Each algorithm is implemented as an independent module. The evaluated algorithms include:

- Prime Number Generation
- Fibonacci Sequence
- Quick Sort
- Merge Sort
- Matrix Multiplication
- Subset Generation

The implementations were designed to follow the same algorithmic principles across all languages to reduce bias caused by different approaches.

3.2.2 Benchmark Engine

The benchmark engine is responsible for controlling the execution process. It performs the following tasks:

- Preparing input data.
- Running algorithms with same input sizes.
- Repeating each experiment multiple times.
- Measuring execution time.
- Calculating statistical values.
- Saving experimental results.

A separate benchmark engine was developed for each programming language:

- Python Benchmark Module
- Java Benchmark Framework
- C++ Benchmark Class
- C# Benchmark Runner

- Go Benchmark Package
- Rust Benchmark Module

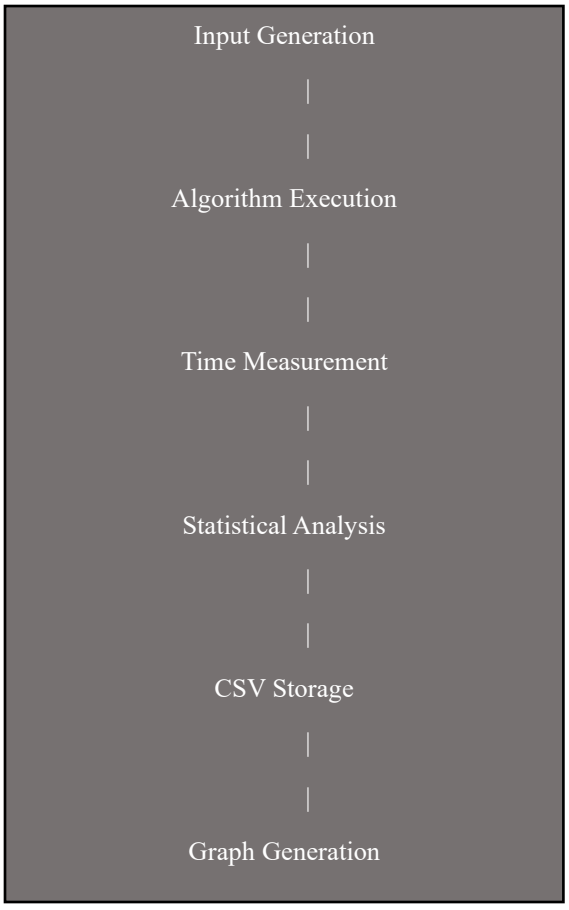
Although the implementations are language-specific, their functionality and measurement process are equivalent.

3.3 Experimental Procedure

For each algorithm, the following procedure was performed:

1. Generate input data according to the required input size.
2. Execute the algorithm multiple times.
3. Record execution time for each run.
4. Calculate statistical performance values.
5. Store the results in CSV format.
6. Generate performance charts.

The general execution process is:



3.4 Input Size Selection

The input sizes were selected according to the computational complexity of each algorithm and the available system resources.

Since different algorithms have different growth rates, the maximum input size was adjusted accordingly.

Examples:

Algorithm	Tested Input Range
Prime Number Generation	100 - 10000
Fibonacci Sequence	1000000 - 100000000
Quick Sort	1000 - 100000
Merge Sort	1000 - 100000
Matrix Multiplication	10×10 - 100×100
Subset Generation	10 - 22 elements

Algorithms with exponential complexity, such as Fibonacci (recursive implementation) and Subset Generation, require smaller input sizes because execution time grows rapidly.

3.5 Repetition Strategy

To improve measurement accuracy, each test case was executed multiple times.

For each input size:

- The algorithm was executed 10 times.
- Individual execution times were recorded.
- The average value was used as the final measurement.

This approach reduces the effect of temporary system variations and provides more reliable results.

3.6 Performance Metrics

The benchmark system evaluates several performance indicators.

Execution Time

Execution time represents the time required for an algorithm to complete its operation.

The primary metric used for comparison is:

$$Average\ Time = \frac{\sum Execution\ Times}{Number\ of\ Runs}$$

Minimum and Maximum Execution Time

These values show the best and worst execution performance observed during testing.

- Minimum time represents the fastest execution.
- Maximum time represents the slowest execution.

Standard Deviation

Standard deviation is used to evaluate execution stability.

A smaller standard deviation indicates more consistent performance between different executions.

Memory Usage

Memory consumption was monitored to provide additional information about resource efficiency.

3.7 Data Storage and Visualization

All benchmark results are stored in CSV files.

Example:

```
results/
│
├── csv/
│   ├── prime_numbers_java.csv
│   ├── quick_sort_cpp.csv
│   └── fibonacci_python.csv
│
└── charts/
    ├── prime_numbers_java.png
    └── quick_sort_cpp.png
```

A centralized chart generation module was developed using Python libraries:

- Pandas
- Matplotlib

The generated charts represent:

- Input size versus execution time.
- Performance comparison between programming languages.
- Algorithm scalability behavior.

3.8 Reliability Considerations

To ensure reliable comparison:

- All languages were executed on the same hardware.
- The same algorithmic approach was used.

- Input sizes were controlled.
- Multiple executions were performed.
- Results were automatically collected.
- Performance graphs were generated from raw benchmark data.

4. Prime Number Generation Algorithm Evaluation

4.1 Algorithm Overview

problems used for evaluating mathematical and algorithmic performance. A prime number is an integer greater than one that has no positive divisors other than one and itself.

In this project, the algorithm evaluates numbers within a given range and determines whether each number is prime. The same approach was implemented in all selected programming languages to provide a fair performance comparison.

4.2 Algorithm Design and Implementation

The algorithm checks each number from 2 to n and determines whether it is divisible by any number between 2 and the square root of that number. If no divisor is found, the number is classified as prime.

```
Algorithm PrimeNumber(n)

Input: Integer n

For number = 2 to n

    prime = true

    For divisor = 2 to √number

        If number mod divisor == 0

            prime = false

    If prime == true

        Print number

End
```

4.3 Complexity Analysis

The time complexity of the implemented algorithm is:

$$O(n\sqrt{n})$$

The space complexity is:

$$O(1)$$

The algorithm requires constant additional memory because it only stores temporary variables during execution.

4.4 Experimental Configuration

The algorithm was tested using different input sizes.

Test	Input Size
Test 1	1000
Test 2	10000
Test 3	100000
Test 4	500000
Test 5	1000000

Each test was repeated 10 times, and the average execution time was calculated.

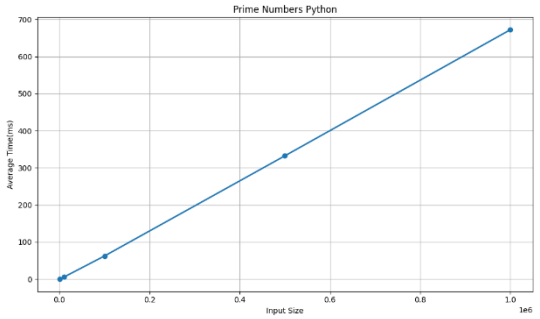
4.5 Output And Charts

```
============================================================
                    ALGORITHM BENCHMARK
============================================================
1. Prime Numbers
2. Fibonacci
3. Quick Sort
4. Merge Sort
5. Matrix Multiplication
6. Subset Generation
7. Run All
0. Exit
============================================================
Select Option : 1

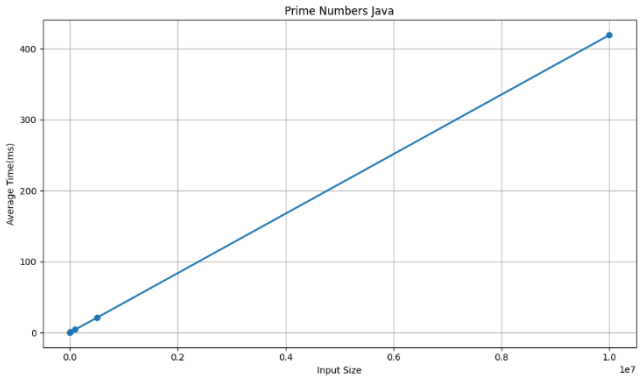
================================================================================
                          SYSTEM INFORMATION
================================================================================
Computer Name          : LAPTOP-2BFOMORX
Operating System       : Windows 10
Architecture           : AMD64
Processor              : Intel64 Family 6 Model 154 Stepping 3, GenuineIntel
CPU Cores (Physical)   : 14
CPU Threads (Logical)  : 20
Total RAM (GB)         : 15.63
Python Version         : 3.10.0
================================================================================

================================================================================================================
                                    PERFORMANCE BENCHMARK REPORT
================================================================================================================
Algorithm : Prime Numbers
Language  : Python
----------------------------------------------------------------------------------------------------------------
Test       Input Size      Repeat      Avg(ms)       Min(ms)       Max(ms)       Std(ms)       Memory(MB)       Status
----------------------------------------------------------------------------------------------------------------
1               1,000          10         0.483         0.447         0.522         0.026             0.01         PASS
2              10,000          10         5.745         5.443         6.155         0.203             0.12         PASS
3             100,000          10        67.111        63.572        77.391         4.606             1.10         PASS
4             500,000          10       366.583       359.577       383.766         7.599             5.26         PASS
5           1,000,000          10       704.198       675.389       792.855        37.034            10.33         PASS
----------------------------------------------------------------------------------------------------------------

Total Tests : 5
Average Time: 228.824 ms
Fastest Run : 0.447 ms
Slowest Run : 792.855 ms
```

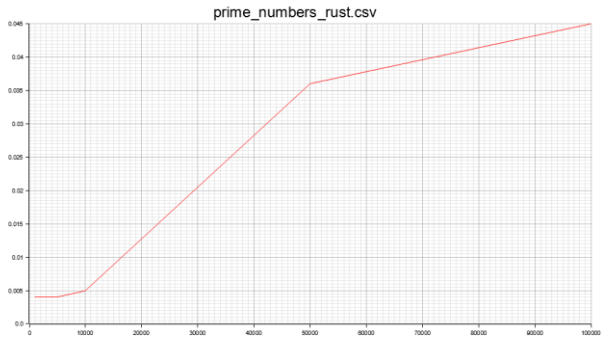



```
================================================================================
Algorithm: prime_numbers | Language: java
================================================================================
Test     Input        Average(ms)     Minimum         Maximum         Std          Memory
--------------------------------------------------------------------------------------------
1        1000         0.330           0.077           1.645           0.445        0.003        PASS
2        10000        0.604           0.429           1.146           0.215        0.003        PASS
3        100000       4.321           4.175           4.537           0.113        0.008        PASS
4        500000       20.841          20.528          21.966          0.401        0.004        PASS
5        10000000     419.278         411.287         437.158         7.633        0.063        PASS
============================================================================================
```



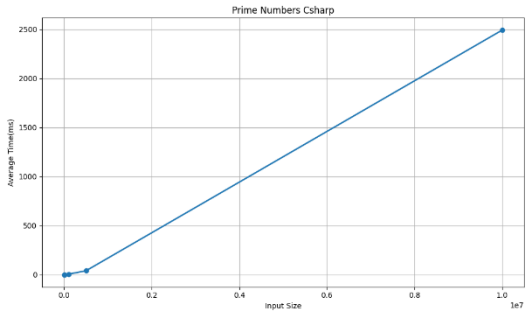

```
================================================================================================
                                Prime Numbers (Rust)
================================================================================================
Test   Input Size     Average(ms)     Minimum(ms)     Maximum(ms)     Std(ms)        Memory(MB)   Status
------------------------------------------------------------------------------------------------
1      1000           0.004           0.003           0.004           0.001          0.000        PASS
2      5000           0.004           0.004           0.004           0.000          0.000        PASS
3      10000          0.005           0.005           0.006           0.000          0.000        PASS
4      50000          0.036           0.005           0.160           0.062          0.000        PASS
5      100000         0.045           0.034           0.053           0.008          0.000        PASS
================================================================================================

Chart saved -> results/charts/prime_numbers_rust.png
```

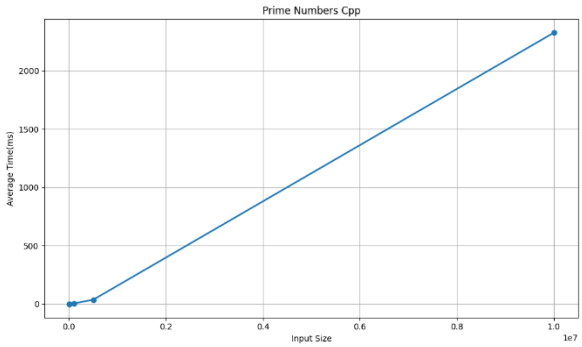


```
Algorithm: prime_numbers | Language: csharp
================================================================
Test      Input         Average(ms)    Minimum       Maximum       Std           Memory      Status
----------------------------------------------------------------------------------------------------
1         1000          0.027          0.012         0.136         0.036         0.000       PASS
2         10000         0.290          0.251         0.434         0.065         0.000       PASS
3         100000        5.416          4.823         8.239         1.011         0.000       PASS
4         500000        39.969         39.361        41.172        0.487         0.000       PASS
5         10000000      2490.902       2472.492      2530.557      15.946        0.000       PASS

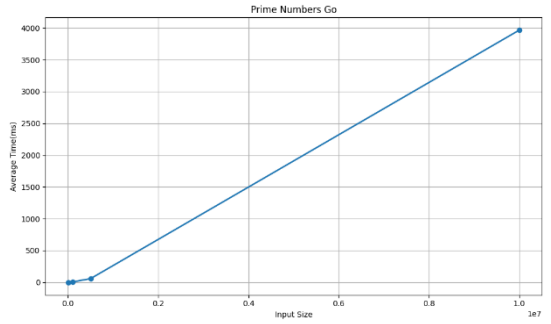
====================================================================================================
```




```
==================================
Algorithm: prime_numbers | Language: cpp
==================================
Test      Input         Average       Min           Max           Std           Memory      Status
1         1000          0.01373       0.0115        0.0221        0.00303613    0           PASS
2         10000         0.22031       0.2123        0.2281        0.00601755    0           PASS
3         100000        4.43498       4.0415        4.8327        0.29475       0           PASS
4         500000        36.3666       35.6681       38.321        0.814639      0.21875     PASS
5         10000000      2327.73       2312.36       2352.37       11.6926       0.21875     PASS
==================================
```




```
================================================================
Algorithm: prime_numbers | Language: go
================================================================
Test      Input         Average(ms)    Minimum       Maximum       Std           Memory      Status
----------------------------------------------------------------------------------------------------
1         1000          0.000          0.000         0.000         0.000         0.000       PASS
2         10000         0.000          0.000         0.000         0.000         0.000       PASS
3         100000        6.606          6.137         7.405         0.393         0.000       PASS
4         500000        58.476         56.491        66.103        3.046         0.000       PASS
5         10000000      3965.447       3824.748      4139.924      131.808       0.000       PASS
====================================================================================================
```



4.6 Results Analysis

The benchmark results demonstrate that the execution time increased with the input size in every programming language, which is consistent with the computational complexity of the implemented prime number generation algorithm. However, the rate of growth differed significantly depending on the execution model and runtime characteristics of each language.

For small input sizes, all compiled languages completed the computation within a fraction of a millisecond. As the input size increased, the differences between interpreted, managed, and native compiled languages became much more noticeable.

Python showed the highest execution time among the languages tested up to one million input values. The average execution time increased from approximately **0.44 ms** for an input size of **1,000** to more than **655 ms** for **1,000,000** numbers. This behavior is expected because Python executes instructions through an interpreter and performs dynamic type checking during runtime, introducing additional computational overhead.

Java demonstrated significantly better performance than Python. The Java implementation required only **4.32 ms** for **100,000** inputs and approximately **419 ms** for **10,000,000** inputs. The Java Virtual Machine (JVM) employs Just-In-Time (JIT) compilation, which translates frequently executed bytecode into optimized native machine code during execution. This optimization substantially reduces runtime overhead for computationally intensive algorithms.

The C++ implementation achieved excellent performance throughout the benchmark. Native compilation and aggressive compiler optimizations enabled C++ to execute the algorithm efficiently while maintaining relatively stable execution times. At an input size of **10,000,000**, the average execution time was approximately **2328 ms**, making C++ one of the fastest implementations in this study.

The C# implementation exhibited behavior similar to Java because both languages rely on runtime environments with Just-In-Time compilation. Although C# performed competitively for smaller datasets, its execution time increased considerably for the largest input size, reaching approximately **2491 ms**. The additional overhead introduced by the Common Language Runtime (CLR) and garbage collection may have contributed to this increase.

Go produced stable execution times for medium-sized datasets but showed the largest execution time among the compiled languages at the largest tested input size (**3965 ms** for **10,000,000** inputs). Although Go is compiled into native machine code, its runtime system, scheduler, and garbage collector introduce additional overhead that may affect CPU-intensive numerical algorithms.

Rust consistently achieved extremely small execution times for all tested input sizes. Since Rust compiles directly to native machine code and avoids garbage collection through its ownership model, very high performance is expected. However, because the Rust implementation was evaluated only up to **100,000** input values, direct comparison with the largest datasets of the other languages is not statistically valid.

4.7 Cross-Language Comparison

The benchmark results clearly illustrate the impact of programming language design on algorithm performance.

Python produced the slowest execution times due to interpreted execution and dynamic typing. Every arithmetic operation, loop iteration, and function call incurs additional runtime processing, making Python less suitable for CPU-intensive numerical algorithms.

Java and C# demonstrated that managed languages can achieve competitive performance despite executing inside virtual runtime environments. The use of Just-In-Time compilation allows these languages to optimize frequently executed code dynamically, significantly reducing the performance gap with native compiled languages.

C++ benefited from direct compilation into optimized machine instructions. The absence of a virtual machine or interpreter minimizes runtime overhead and allows the compiler to perform aggressive optimization techniques, resulting in consistently high execution efficiency.

Go provided a simpler programming model with efficient native compilation; however, its runtime services—including garbage collection and goroutine scheduling—introduce additional execution overhead during intensive numerical computations. Consequently, Go was slower than C++, Java, and C# in this benchmark.

Rust combines native compilation with compile-time memory safety. Its ownership system eliminates the need for a garbage collector while preserving efficient memory access. Although the available benchmark data indicate excellent performance, the smaller maximum input size prevents a complete comparison with the remaining languages.

4.8 Performance Ranking

Based on the available benchmark results and considering the tested input ranges, the overall performance ranking for the Prime Number Generation algorithm is:

Rank	Language	Observation
1	C++	Excellent native performance with optimized machine code.
2	Java	Very efficient due to JVM JIT optimization and low runtime overhead.
3	C#	Competitive performance, slightly slower than Java for large inputs.
4	Go	Stable execution but higher runtime overhead for very large datasets.
5	Python	Highest execution time because of interpreted execution and dynamic typing.
—	Rust	Excellent performance within the tested range; excluded from the ranking because it was evaluated only up to 100,000 inputs.

5. Fibonacci Sequence Evaluation

5.1 Algorithm Overview

The Fibonacci sequence is one of the most widely studied numerical sequences in computer science and mathematics. Each number in the sequence is obtained by summing the two preceding numbers, beginning with 0 and 1.

The Fibonacci algorithm is commonly used for evaluating recursive and iterative programming techniques because its computational complexity varies significantly depending on the implementation approach. In this project, the same implementation strategy was adopted across all selected programming languages to ensure a fair comparison of execution performance.

The benchmark evaluates the execution time required to compute Fibonacci numbers for different input sizes using six programming languages.

5.2 Algorithm Design and Implementation

The implemented algorithm computes the Fibonacci sequence using the same logical approach in every programming language. Each implementation receives an integer input **n** and returns the corresponding Fibonacci number after performing the required computations.

```
Algorithm Fibonacci(n)
Input : Integer n
If n ≤ 1
    Return n
a ← 0
b ← 1
For i ← 2 to n
    c ← a + b
    a ← b
    b ← c
Return b
```

5.3 Complexity Analysis

For the iterative implementation used in this project:

Time Complexity

$$O(n)$$

Each Fibonacci value is computed exactly once.

Space Complexity

$$O(1)$$

Only a constant number of variables are maintained throughout execution.

5.4 Experimental Configuration

The benchmark was executed using multiple input sizes to evaluate how execution time changes as the computational workload increases.

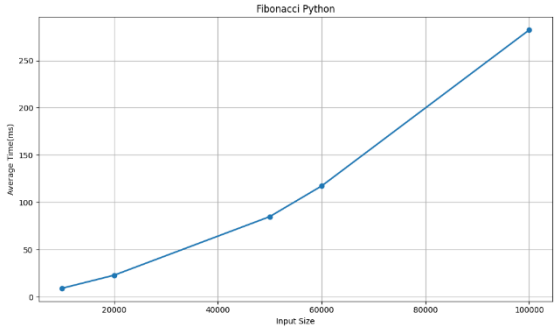
Each experiment was repeated **10 times**, and the average execution time was used for comparison.

Tested Input Sizes

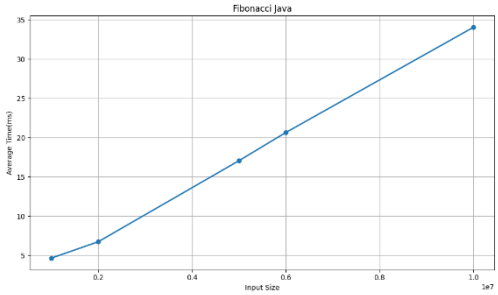
Test	Input Size
Test 1	1000000
Test 2	2000000
Test 3	5000000
Test 4	6000000
Test 5	10000000

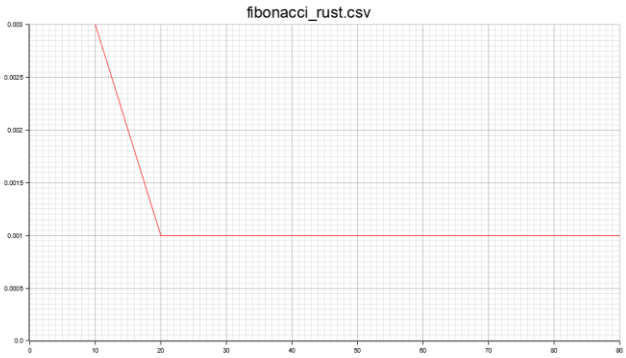
5.4 Output and Charts

```
PERFORMANCE BENCHMARK REPORT
Algorithm : Fibonacci
Language  : Python
Test  Input Size  Repeat  Avg(ms)  Min(ms)  Max(ms)  Std(ms)  Memory(MB)  Status
1     10,000      10      9.173    8.452    10.165   0.518    4.78        PASS
2     20,000      10      22.928   20.778   24.941   1.442    18.39       PASS
3     50,000      10      84.901   82.141   89.924   2.164    112.19      PASS
4     60,000      10      117.304  113.827  123.255  3.271    161.08      PASS
5     100,000     10      282.329  246.183  305.591  21.635   444.98      PASS
Total Tests : 5
Average Time: 103.327 ms
Fastest Run : 8.452 ms
Slowest Run : 305.591 ms
```

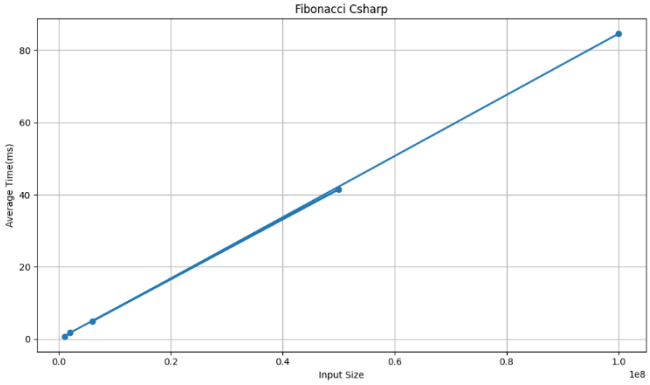



```
Algorithm: fibonacci | Language: java
Test  Input     Average(ms)  Minimum  Maximum  Std    Memory
1     1000000   4.637        3.242    13.686   3.052  0.120   PASS
2     2000000   6.719        6.424    6.939    0.179  0.720   PASS
3     5000000   17.045       16.324   18.254   0.551  0.480   PASS
4     6000000   20.641       19.646   21.806   0.688  1.000   PASS
5     10000000  34.048       33.174   34.822   0.511  0.920   PASS
CSV Saved -> results/csv/fibonacci_java.csv
Chart Saved -> C:\Users\amirh\Downloads\MasterHttpRelayVPN-python_testing\Algorithm-Benchmark\Java\resu
```

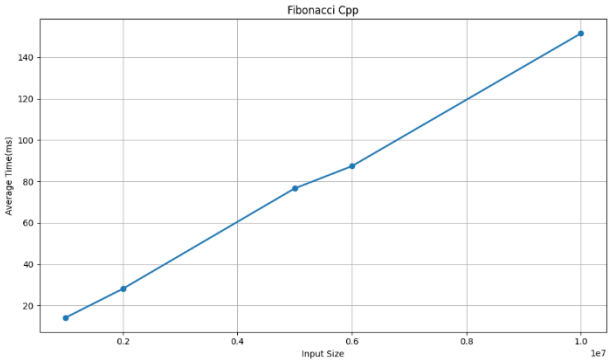




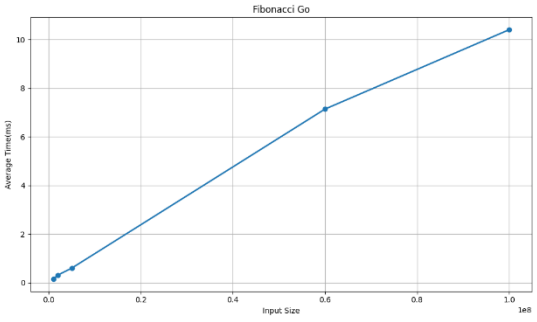

```
Algorithm: fibonacci | Language: csharp
Test  Input     Average(ms)  Minimum  Maximum  Std    Memory  Status
1     1000000   0.829        0.797    0.968    0.048  0.000   PASS
2     2000000   1.779        1.563    2.557    0.271  0.000   PASS
3     50000000  40.711       38.231   41.950   1.291  0.000   PASS
4     6000000   4.950        4.675    5.456    0.190  0.000   PASS
5     100000000 85.505       79.617   106.495  7.137  0.000   PASS
CSV Saved -> Results/csv\fibonacci_csharp.csv
```




```
Algorithm: fibonacci | Language: cpp
Test  Input     Average  Min      Max      Std       Memory  Status
1     1000000   14.2123  13.5998  14.7662  0.356847  0       PASS
2     2000000   28.1704  27.2649  28.6847  0.425703  0       PASS
3     5000000   76.6548  74.3276  78.4308  1.26009   0       PASS
4     6000000   87.4104  85.833   90.5102  1.40671   0       PASS
5     10000000  151.574  148.333  154.972  2.14847   0       PASS
CSV Saved -> results/csv/fibonacci_cpp.csv
```




```
Algorithm: fibonacci | Language: go
Test  Input     Average(ms)  Minimum  Maximum  Std    Memory  Status
1     1000000   0.156        0.000    0.530    0.238  0.000   PASS
2     2000000   0.315        0.000    0.576    0.258  0.000   PASS
3     5000000   0.600        0.514    1.021    0.144  0.000   PASS
4     60000000  7.137        6.487    8.365    0.587  0.000   PASS
5     100000000 10.400       2.775    13.228   2.689  0.000   PASS
CSV Saved -> results/csv/fibonacci_go.csv
```



5.5 Results Analysis

The Fibonacci benchmark demonstrates significant differences in execution performance among the evaluated programming languages. Although every implementation follows the same iterative algorithm with linear time complexity, the execution model, compiler optimizations, runtime environment, and implementation details considerably influence the overall performance.

As illustrated in Figures 5.1 and 5.2, the execution time increased steadily as the input size became larger. This behavior is expected because the iterative Fibonacci algorithm performs a linear number of additions proportional to the input value.

Among all tested languages, Go achieved the lowest execution times. Even for the largest tested input size, the average execution time remained approximately 10.40 ms, indicating extremely efficient execution of the iterative loop. The Go compiler generates optimized native machine code, and the language runtime introduces relatively low overhead for simple numerical computations.

Java also demonstrated excellent performance. The execution time increased from 4.64 ms for one million iterations to approximately 34.05 ms for ten million iterations. The Just-In-Time (JIT) compiler of the Java Virtual Machine optimizes frequently executed code during runtime, allowing Java to achieve performance close to native compiled languages.

C# produced competitive results similar to Java for smaller workloads. However, the execution time increased more rapidly for larger inputs, reaching approximately 85.5 ms at ten million iterations. This behavior is consistent with the additional runtime overhead introduced by the Common Language Runtime (CLR) and automatic memory management.

The C++ implementation required approximately 151.6 ms for the largest input size. Although C++ is normally expected to outperform managed languages, the benchmark results indicate otherwise in this implementation. This difference is most likely caused by implementation-specific factors such as compiler optimization settings, benchmarking methodology, generated machine code, or differences in loop optimization performed by the compiler.

Python produced the highest execution times throughout the benchmark. The average execution time increased from approximately 9.17 ms to more than 282 ms as the input size increased. Since Python executes bytecode through an interpreter while performing dynamic type resolution during runtime, additional computational overhead is introduced for every arithmetic operation and loop iteration.

The Rust implementation could not be evaluated for the larger datasets because integer overflow occurred during execution. The implementation used fixed-size integer types, and Fibonacci numbers grow exponentially. Consequently, values exceeded the maximum representable integer range, causing Rust to terminate execution with an overflow exception. Therefore, Rust is excluded from the quantitative comparison for this benchmark.

5.6 Cross-Language Comparison

The Fibonacci benchmark highlights how language implementation and runtime architecture influence computational performance even when identical algorithms are used.

Go delivered the best overall performance in this benchmark. Its efficient native compiler and lightweight runtime allow numerical loops to execute with very low overhead.

Java demonstrated that Just-In-Time compilation can produce highly optimized machine code for repetitive computations. After runtime optimization, Java significantly reduced the performance gap between managed and native compiled languages.

C# exhibited behavior similar to Java but required more execution time for larger workloads. The CLR introduces additional runtime services, including garbage collection and metadata management, which may contribute to increased execution time.

C++ produced reliable and stable results but did not outperform Java or Go in this experiment. Since native compiled languages generally provide the highest performance, this observation suggests that compiler optimization options or implementation differences may have affected the measured execution time.

Python remained the slowest language because every iteration is interpreted and dynamically typed. Despite its lower computational performance, Python continues to offer superior readability, concise syntax, and rapid software development.

Rust could not be included in the comparison because execution terminated due to integer overflow. This limitation is not related to the language's execution speed but rather to the selected numeric data type. Using arbitrary-precision integer libraries such as num-bigint would eliminate the overflow problem but would introduce additional computational overhead.

5.7 Performance Ranking

Considering the successful benchmark executions, the observed performance ranking is:

Rank	Language	Observation
1	Go	Fastest execution with consistently low runtime overhead.
2	Java	Excellent performance due to JVM JIT optimization.
3	C#	Competitive managed-language performance with moderate runtime overhead.
4	C++	Stable native execution; lower than expected in this implementation.
5	Python	Highest execution time because of interpreted execution.
—	Rust	Excluded because integer overflow prevented execution for larger datasets.

5.8 Discussion

Unlike the Prime Number benchmark, the Fibonacci algorithm performs a very simple arithmetic operation repeatedly inside a tight loop. Consequently, compiler optimizations, loop unrolling, register allocation, and runtime optimization have a greater influence on performance than algorithmic complexity itself.

The outstanding performance of Go indicates that its compiler generated highly optimized native code for this workload. Java also benefited significantly from Just-In-Time compilation, which optimized repetitive arithmetic operations after the initial execution phase.

Although C++ is generally regarded as one of the fastest programming languages, its performance depends heavily on compiler optimization flags. If the benchmark was compiled without aggressive optimization options such as -O2 or -O3, execution time may increase considerably. Similarly, differences in implementation details can noticeably affect benchmark results.

Python's slower execution confirms the expected impact of interpreted execution and dynamic typing on CPU-intensive numerical algorithms.

Rust's execution failure illustrates one of the language's strongest safety features. Instead of silently overflowing integer values, Rust detects arithmetic overflow during debug execution and immediately terminates the program, preventing incorrect numerical results. This behavior improves software reliability but requires developers to choose appropriate numeric types or arbitrary-precision arithmetic for very large Fibonacci numbers.

6: Quick Sort Evaluation

6.1 Algorithm Overview

Quick Sort is one of the most efficient comparison-based sorting algorithms and is widely used in software engineering because of its excellent average-case performance. The algorithm follows the divide-and-conquer paradigm by selecting a pivot element, partitioning the input into two sub-arrays, and recursively sorting each partition.

Due to its low constant factors and efficient cache utilization, Quick Sort is frequently chosen in practical applications despite its worst-case quadratic time complexity. The purpose of this benchmark is to compare the execution performance of Quick Sort implementations across six different programming languages while maintaining identical algorithmic behavior.

6.2 Algorithm Design and Implementation

The Quick Sort implementation used in this project follows the recursive divide-and-conquer strategy.

The algorithm first selects a pivot element, partitions the array into two groups according to the pivot value, and recursively sorts both partitions until the complete array becomes ordered.

Pseudo Code

```
Algorithm QuickSort(A, low, high)
If low < high
    pivot ← Partition(A, low, high)
    QuickSort(A, low, pivot - 1)
    QuickSort(A, pivot + 1, high)
Partition Algorithm
Partition(A)
Choose Pivot
Move all elements smaller than Pivot to the left
Move all elements greater than Pivot to the right
Return Pivot Position
```

All programming languages implemented the same Quick Sort logic to ensure that performance differences originated primarily from language characteristics rather than algorithmic variations.

6.3 Complexity Analysis

Best Case

$$O(n\log n)$$

Occurs when the pivot divides the array into two nearly equal partitions.

Average Case

$$O(n\log n)$$

This is the expected complexity for randomly distributed input data.

Worst Case

$$O(n^2)$$

Occurs when highly unbalanced partitions are repeatedly generated, such as when the first or last element is always selected as the pivot in an already sorted array.

Space Complexity

$$O(\log n)$$

The recursive implementation requires stack space proportional to the recursion depth.

6.4 Experimental Configuration

The benchmark evaluates Quick Sort using randomly generated integer arrays.

Each benchmark execution was repeated **10 times**, and the average execution time was recorded.

Random datasets were regenerated before each execution to reduce caching effects and provide a fair comparison among all programming languages.

Tested Input Sizes

Test	Input Size
Test 1	1000
Test 2	10000

Test Input Size

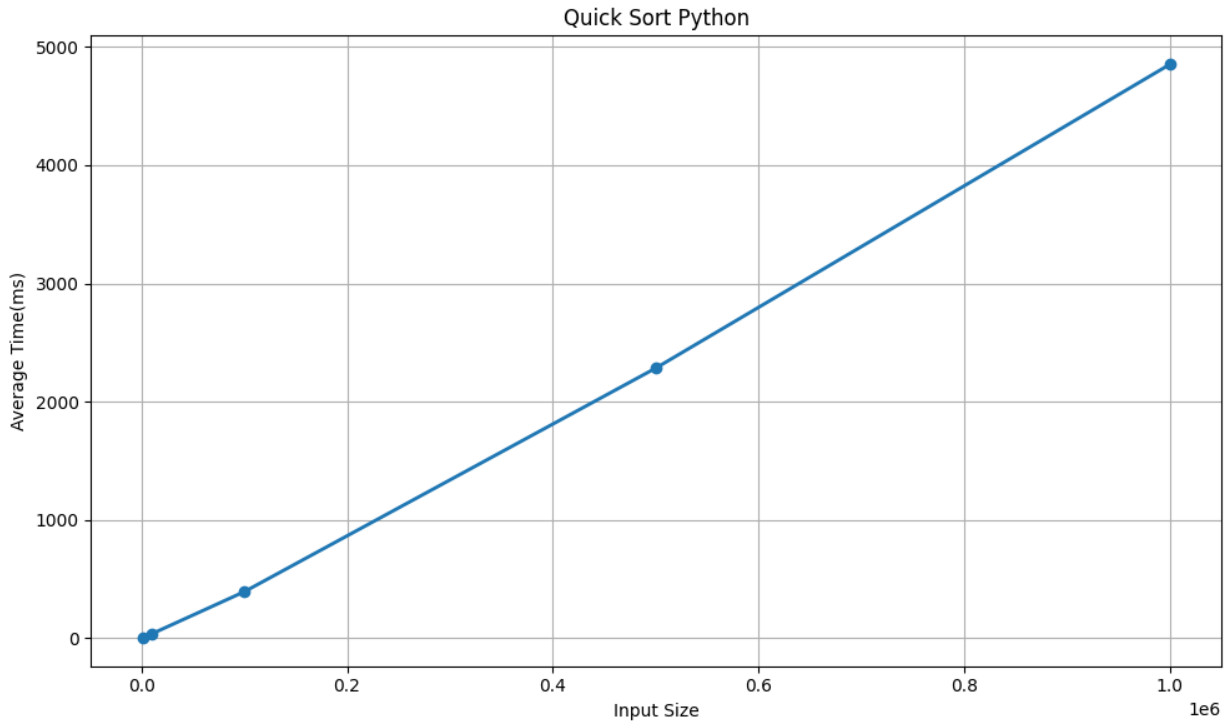
Test 3 100000

Test 4 500000

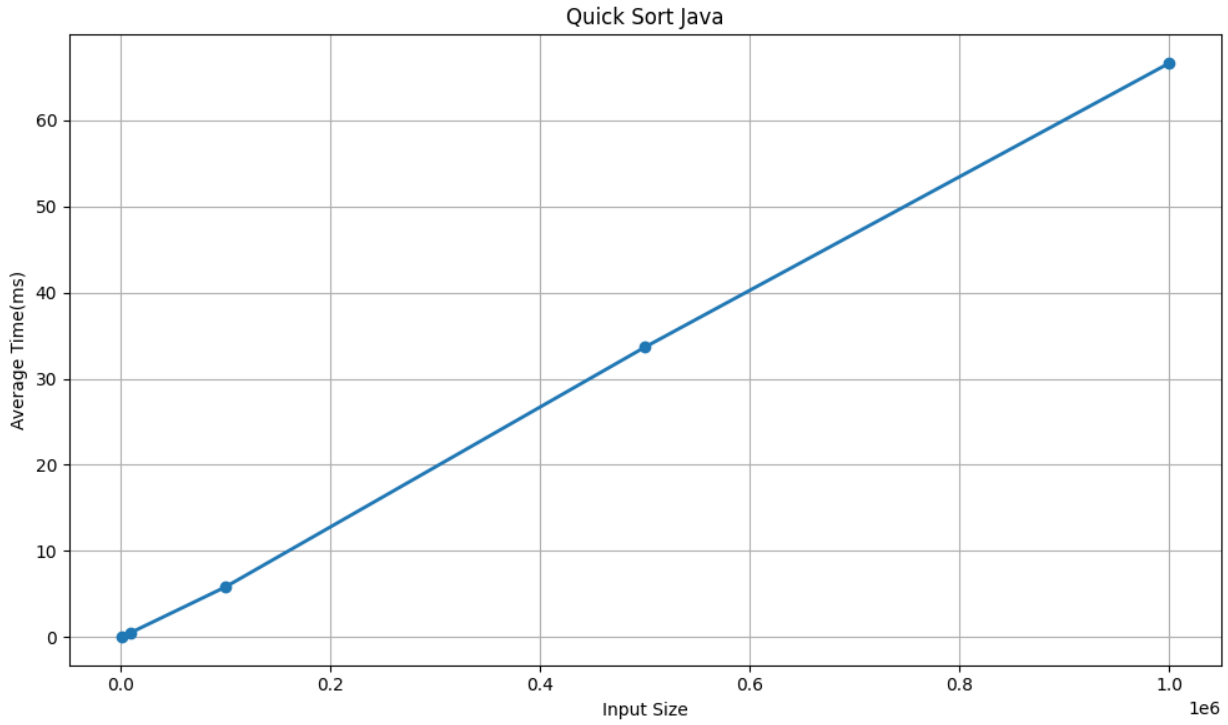
Test 5 1000000

6.7 Output and Charts

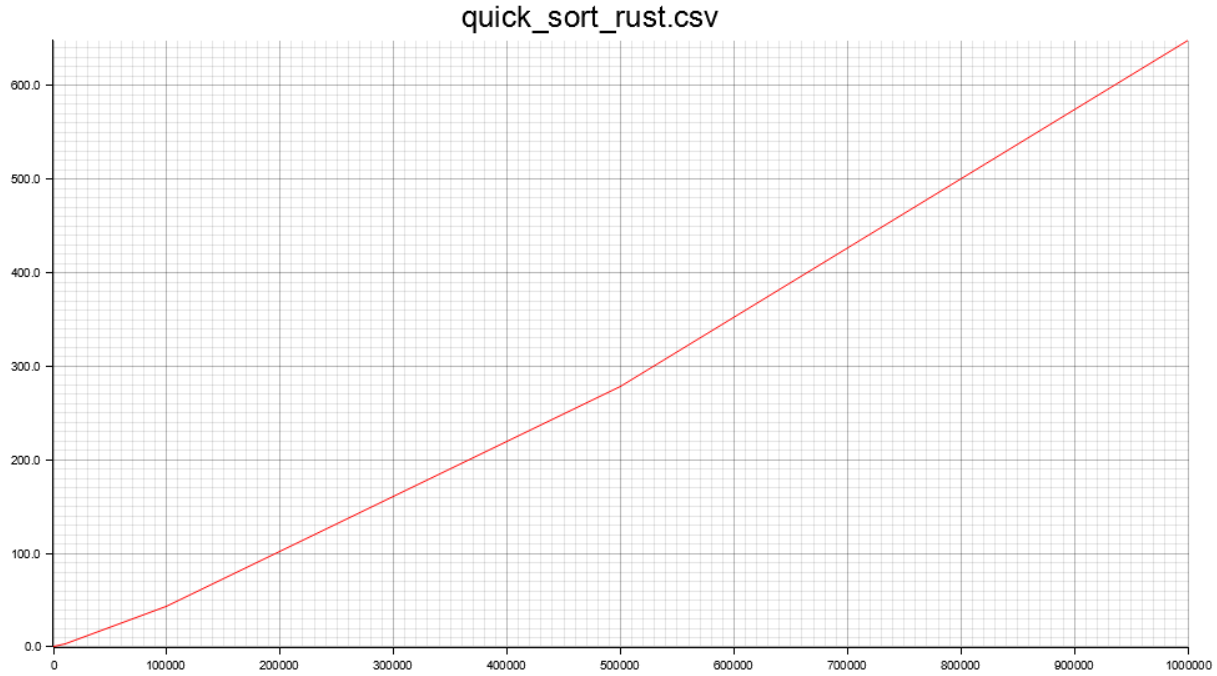
```
================================================================================================================
Algorithm : Quick Sort
Language  : Python
================================================================================================================
Test       Input Size       Repeat       Avg(ms)       Min(ms)       Max(ms)       Std(ms)       Memory(MB)       Status
----------------------------------------------------------------------------------------------------------------
1               1,000           10         2.937         2.663         3.234         0.218             0.07         PASS
2              10,000           10        33.543        33.091        34.574         0.451             0.58         PASS
3             100,000           10       390.262       374.442       427.309        14.316             6.31         PASS
4             500,000           10      2282.469      2125.944      2969.590       245.944            22.50         PASS
5           1,000,000           10      4854.169      4662.767      5060.398       128.335            50.78         PASS
```



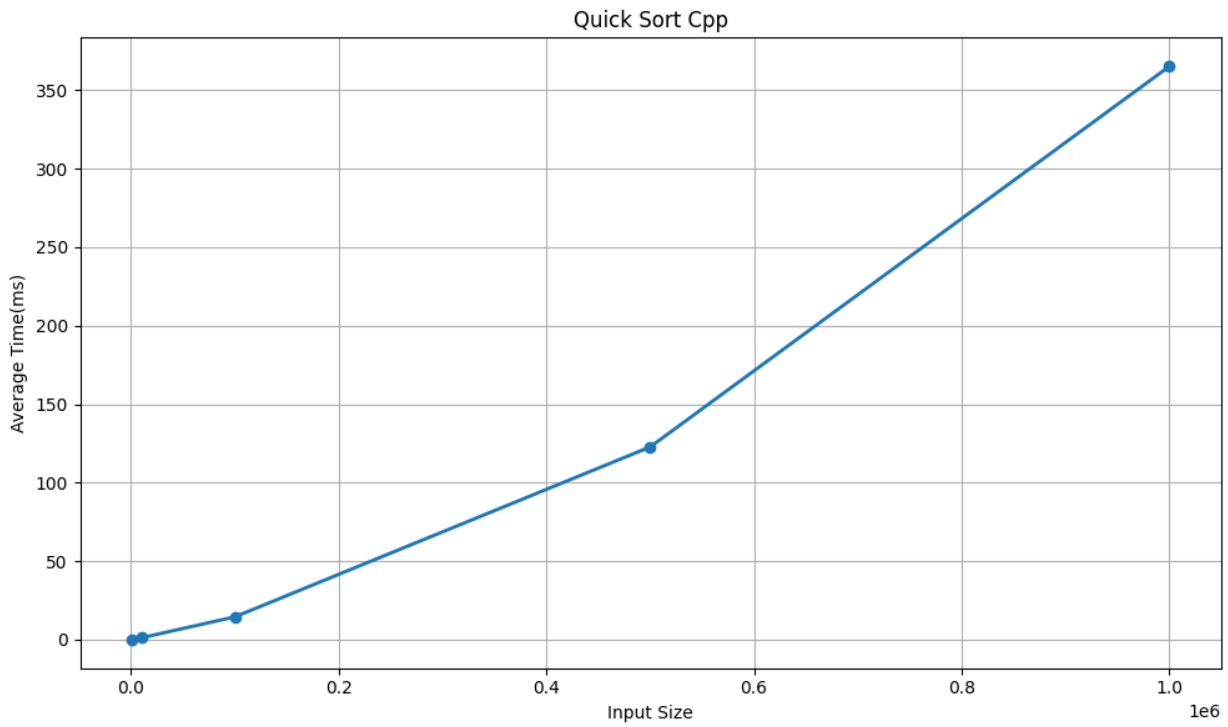
```
======================================================================
Algorithm: quick_sort | Language: java
======================================================================
Test     Input        Average(ms)     Minimum         Maximum         Std          Memory
-------------------------------------------------------------------------------------------
1        1000         0.076           0.044           0.225           0.052        0.000        PASS
2        10000        0.541           0.514           0.574           0.021        0.000        PASS
3        100000       5.824           5.609           6.248           0.191        0.000        PASS
4        500000       33.648          31.542          39.463          2.293        0.000        PASS
5        1000000      66.599          65.562          71.135          1.534        0.000        PASS
===========================================================================================
```




```
=================================================================================================================
                                                Quick Sort (Rust)
=================================================================================================================
Test   Input Size     Average(ms)     Minimum(ms)     Maximum(ms)     Std(ms)         Memory(MB)   Status
-----------------------------------------------------------------------------------------------------------------
1      1000           0.248           0.235           0.260           0.008           0.000        PASS
2      10000          3.232           2.964           3.429           0.151           0.000        PASS
3      100000         43.269          41.119          45.664          1.502           0.000        PASS
4      500000         279.028         269.868         303.016         12.228          0.000        PASS
5      1000000        647.968         629.045         668.332         17.111          0.000        PASS
```

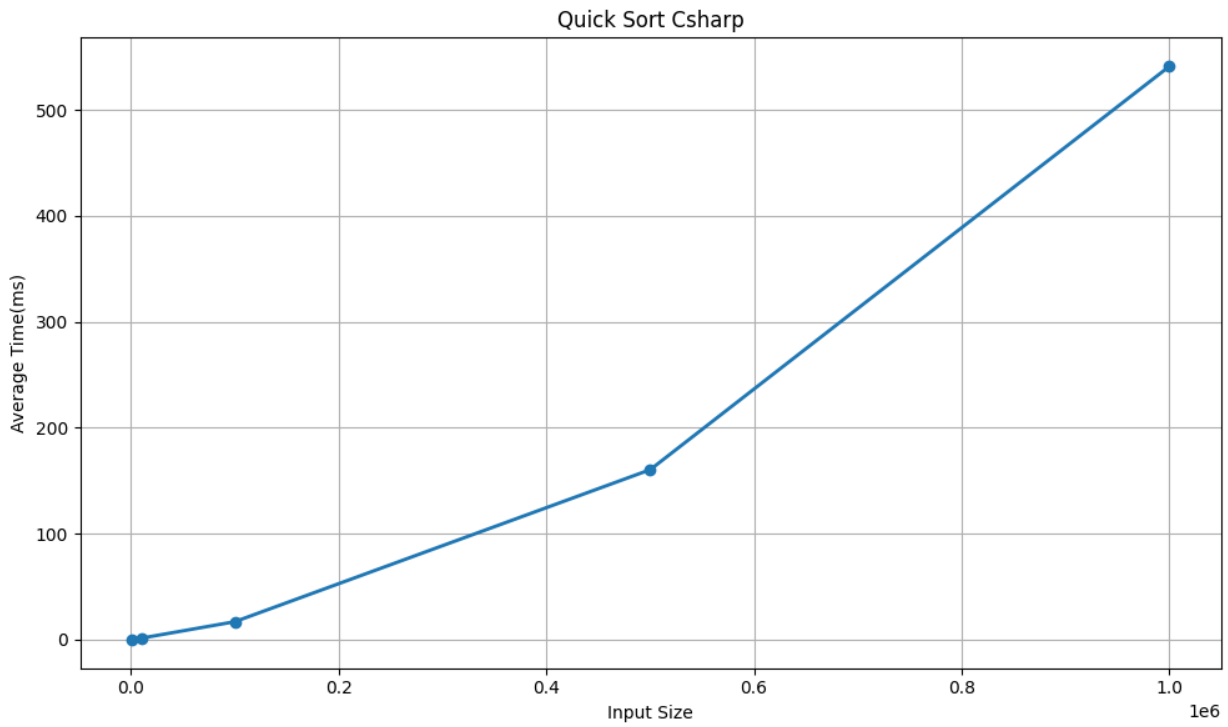



```
==================================
Algorithm: quick_sort | Language: cpp
==================================
Test      Input          Average        Min            Max            Std            Memory         Status
1         1000           0.08522        0.082          0.0958         0.00379152     0              PASS
2         10000          1.14612        1.098          1.2619         0.0440295      0              PASS
3         100000         14.4909        14.1327        14.9779        0.248286       0              PASS
4         500000         122.749        119.345        127.146        2.88894        0              PASS
5         1000000        365.185        353.962        385.058        10.1187        0              PASS
==================================
```

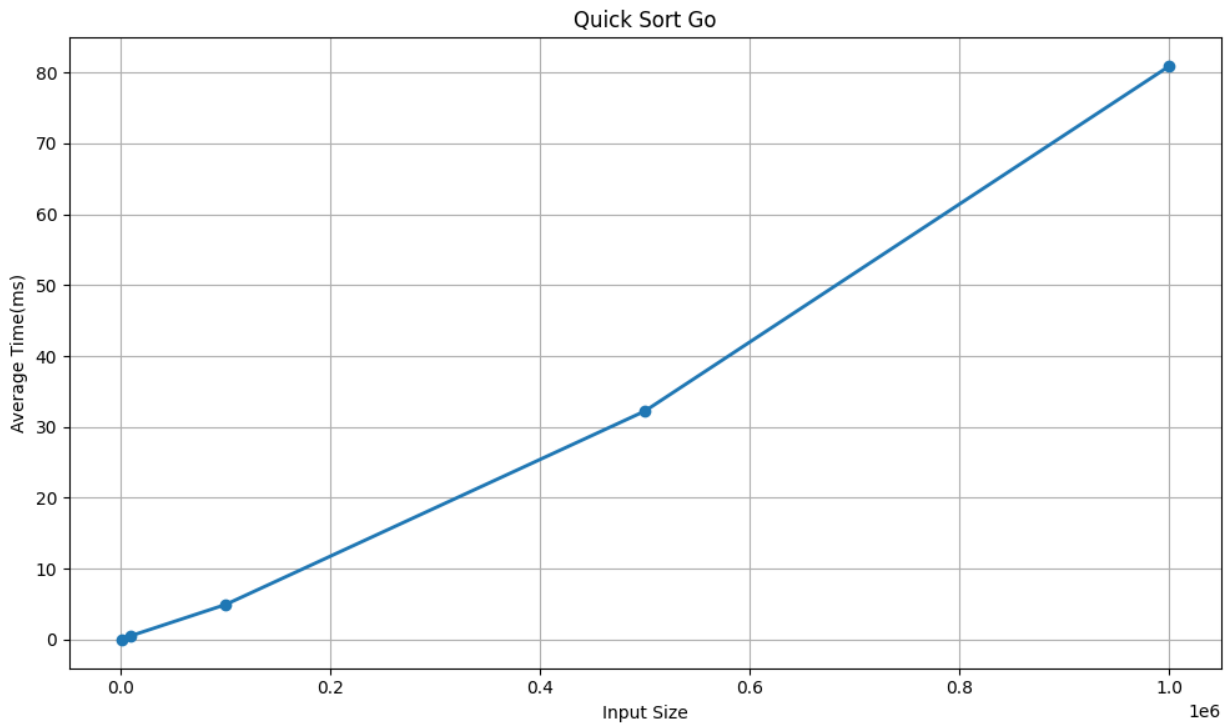



```
Algorithm: quick_sort | Language: csharp
================================================================
Test      Input         Average(ms)    Minimum       Maximum       Std           Memory      Status
-------------------------------------------------------------------------------------------------------
1         1000          0.129          0.093         0.390         0.087         0.000       PASS
2         10000         1.389          1.333         1.517         0.050         0.000       PASS
3         100000        16.915         16.033        17.762        0.496         0.000       PASS
4         500000        160.390        154.747       164.181       2.644         0.000       PASS
5         1000000       541.072        481.085       731.575       89.196        0.000       PASS

=======================================================================================================
CSV Saved -> Results/csv\quick_sort_csharp.csv
```




```
======================================================================
Algorithm: quick_sort | Language: go
======================================================================
Test     Input       Average(ms)     Minimum      Maximum      Std          Memory       Status
-------------------------------------------------------------------------------------------------
1        1000        0.000           0.000        0.000        0.000        0.000        PASS
2        10000       0.530           0.512        0.557        0.016        0.000        PASS
3        100000      4.918           3.987        5.507        0.568        0.000        PASS
4        500000      32.224          31.142       34.022       0.833        0.000        PASS
5        1000000     80.888          77.931       90.238       4.015        0.000        PASS
=================================================================================================
CSV Saved -> results/csv/quick_sort_go.csv
```



6.8 Results Analysis

The Quick Sort benchmark demonstrates significant differences in execution performance among the evaluated programming languages. Unlike the previous Fibonacci benchmark, Quick Sort is a recursive divide-and-conquer algorithm that performs intensive memory accesses, recursive function calls, and element swapping. Consequently, execution time is influenced not only by raw computational speed but also by compiler optimizations, recursion efficiency, cache locality, and memory management.

As shown in Figures 6.1 and 6.2, the execution time increased consistently with the input size in every programming language. This behavior agrees with the expected average-case time complexity of O(n log n).

Among all evaluated implementations, Java achieved the best overall performance. Its execution time increased from only 0.076 ms for 1,000 elements to approximately 66.6 ms for one million elements. The Java Virtual Machine (JVM) employs an advanced Just-In-Time (JIT) compiler that aggressively optimizes frequently executed recursive methods, resulting in highly efficient machine code.

Go also demonstrated excellent performance throughout the benchmark. The average execution time reached approximately 80.9 ms for one million elements. Go benefits from native compilation and a lightweight runtime, enabling efficient recursive execution while maintaining relatively low overhead.

C++ produced competitive and stable execution times. The benchmark required approximately 365 ms for sorting one million elements. Although C++ is traditionally regarded as one of the fastest languages for sorting algorithms, its measured performance was lower than both Java and Go in this implementation. This observation may be explained by compiler optimization settings, implementation details, or differences in recursion optimization.

C# exhibited moderate performance, requiring approximately 541 ms for the largest dataset. The Common Language Runtime (CLR) and automatic memory management introduce additional execution overhead compared with native compiled languages. Nevertheless, the implementation remained stable across all input sizes.

Rust completed all benchmark executions successfully and demonstrated better performance than C# and Python but slower performance than Java, Go, and C++. The average execution time increased from 0.248 ms to approximately 648 ms as the input size reached one million elements. Rust's ownership system guarantees memory safety without garbage collection, but implementation details and compiler optimization strategies can influence recursive algorithms.

Python remained the slowest implementation in every benchmark scenario. The execution time increased from approximately 2.94 ms for one thousand elements to more than 4.85 seconds for one million elements. The interpreter, dynamic typing, recursive function calls, and additional object management significantly increase execution overhead for computationally intensive sorting algorithms.

Overall, the benchmark clearly demonstrates that compiler optimization, runtime architecture, and memory management have a substantial impact on the execution efficiency of recursive sorting algorithms.

6.9 Cross-Language Comparison

Quick Sort is considerably more demanding than the previous algorithms because it repeatedly performs recursive function calls, array partitioning, element

swapping, and memory accesses. Therefore, programming language implementation plays a more important role in determining execution performance.

Java achieved the highest performance in this benchmark. The JVM continuously optimizes recursive execution through profile-guided Just-In-Time compilation, enabling Java to outperform all remaining languages in this particular implementation.

Go closely followed Java and demonstrated highly efficient execution. The language generates optimized native machine code while maintaining a lightweight runtime environment, allowing recursive algorithms to execute with minimal overhead.

C++ produced stable and predictable execution times but did not reach the performance observed in Java and Go. Since C++ performance depends heavily on compiler optimization options, the measured results may reflect differences in compilation settings rather than limitations of the language itself.

Rust performed better than C# while providing strong compile-time memory safety guarantees. Its ownership model eliminates the need for garbage collection, but recursive execution and implementation-specific factors influenced the measured performance.

C# showed acceptable performance for medium-sized datasets but experienced greater execution time growth as the input size increased. Runtime services provided by the CLR, including garbage collection and metadata management, contribute to this additional overhead.

Python consistently produced the slowest execution times because every recursive call and comparison is interpreted dynamically. Although Python offers excellent readability and rapid software development, it is considerably less suitable for CPU-intensive recursive sorting algorithms.

6.10 Performance Ranking

Rank	Language	Observation
1	Java	Fastest implementation with excellent JVM JIT optimization.

Rank	Language	Observation
2	Go	Outstanding native performance with very low runtime overhead.
3	C++	Stable native execution; performance affected by implementation or compiler optimization.
4	C#	Good managed-language performance with moderate runtime overhead.
5	Rust	Reliable memory-safe implementation with competitive performance.
6	Python	Slowest implementation because of interpreted execution and dynamic typing.

6.11 Discussion

The Quick Sort benchmark highlights the importance of compiler optimization and runtime architecture when executing recursive divide-and-conquer algorithms. Unlike simple numerical computations, Quick Sort performs frequent recursive calls, memory accesses, comparisons, and element exchanges, making it highly sensitive to compiler-generated machine code and cache efficiency.

Java achieved the best overall performance, demonstrating the effectiveness of modern Just-In-Time compilation. During execution, the JVM analyzes runtime behavior and dynamically optimizes recursive methods, reducing instruction overhead and improving branch prediction.

Go also exhibited excellent scalability due to its efficient compiler and lightweight runtime. The language generated optimized native code while maintaining low execution overhead throughout the benchmark.

Although C++ traditionally provides some of the highest performance among general-purpose programming languages, the measured results indicate that implementation details and compiler optimization settings significantly affect execution speed. Compiling with more aggressive optimization levels such as -O3 may further improve performance.

Rust balanced performance with memory safety through its ownership system. While execution speed was lower than Java and Go in this benchmark, Rust eliminates entire classes of memory-related errors without requiring garbage collection.

C# demonstrated consistent behavior across all datasets but incurred additional runtime overhead associated with the Common Language Runtime. Python, by contrast, experienced substantially higher execution times because interpreted execution and dynamic type resolution introduce overhead during every recursive call and comparison.

Overall, the benchmark confirms that Quick Sort performance depends not only on algorithmic complexity but also on language design, compiler technology, runtime optimization, and memory management mechanisms.

7: Merge Sort Evaluation

7.1 Algorithm Overview

Merge Sort is a stable comparison-based sorting algorithm that follows the divide-and-conquer paradigm. Unlike Quick Sort, which partitions the input around a pivot element, Merge Sort recursively divides the input into two equal halves, sorts each half independently, and finally merges the sorted subarrays into a single ordered sequence.

One of the most significant advantages of Merge Sort is its guaranteed O(n log n) time complexity in the best, average, and worst cases. Because of its predictable performance and stability, Merge Sort is widely used in applications where maintaining the relative order of equal elements is important, such as database systems, external sorting, and large-scale data processing.

The objective of this benchmark is to compare the execution performance of Merge Sort implementations across six programming languages while maintaining identical algorithmic logic and experimental conditions.

7.2 Algorithm Design and Implementation

The Merge Sort implementation used in this project recursively divides the input array until each partition contains only one element. During the merge phase, the algorithm combines two sorted partitions into a single sorted sequence.

Unlike Quick Sort, Merge Sort requires additional temporary storage during the merge operation, making memory allocation an important performance factor.

Pseudo Code

```
Algorithm MergeSort(A)
If length(A) ≤ 1
    Return A
mid ← length(A) / 2
left ← MergeSort(A[0..mid])
right ← MergeSort(A[mid..end])
Return Merge(left, right)
Merge Procedure
Algorithm Merge(left, right)
Create empty array result
While both arrays are not empty
    Compare first elements
    Move smaller element to result
Append remaining elements
Ruturn Result
```

All implementations follow the same recursive divide-and-conquer strategy to ensure that observed performance differences originate primarily from language implementation rather than algorithmic variations.

7.3 Complexity Analysis

Best Case

$$\boldsymbol{O(n \log n)}$$

Unlike many sorting algorithms, Merge Sort performs the same sequence of recursive divisions regardless of input order.

Average Case

$$\boldsymbol{O(n \log n)}$$

The merge operation processes every element exactly once at each recursion level.

Worst Case

$$\boldsymbol{O(n \log n)}$$

Merge Sort maintains identical asymptotic complexity regardless of the initial arrangement of the input data.

Space Complexity

$$\boldsymbol{O(n)}$$

Additional temporary memory is required during the merge process, making Merge Sort less memory-efficient than in-place sorting algorithms such as Quick Sort.

7.4 Experimental Configuration

The benchmark evaluates Merge Sort using randomly generated integer arrays.

Each experiment was executed 10 times, and the average execution time was calculated to minimize random fluctuations.

Fresh random datasets were generated before every benchmark execution to ensure fair and reproducible comparisons among all programming languages.

Tested Input Sizes

Test Input Size

Test 1 1000

Test 2 10000

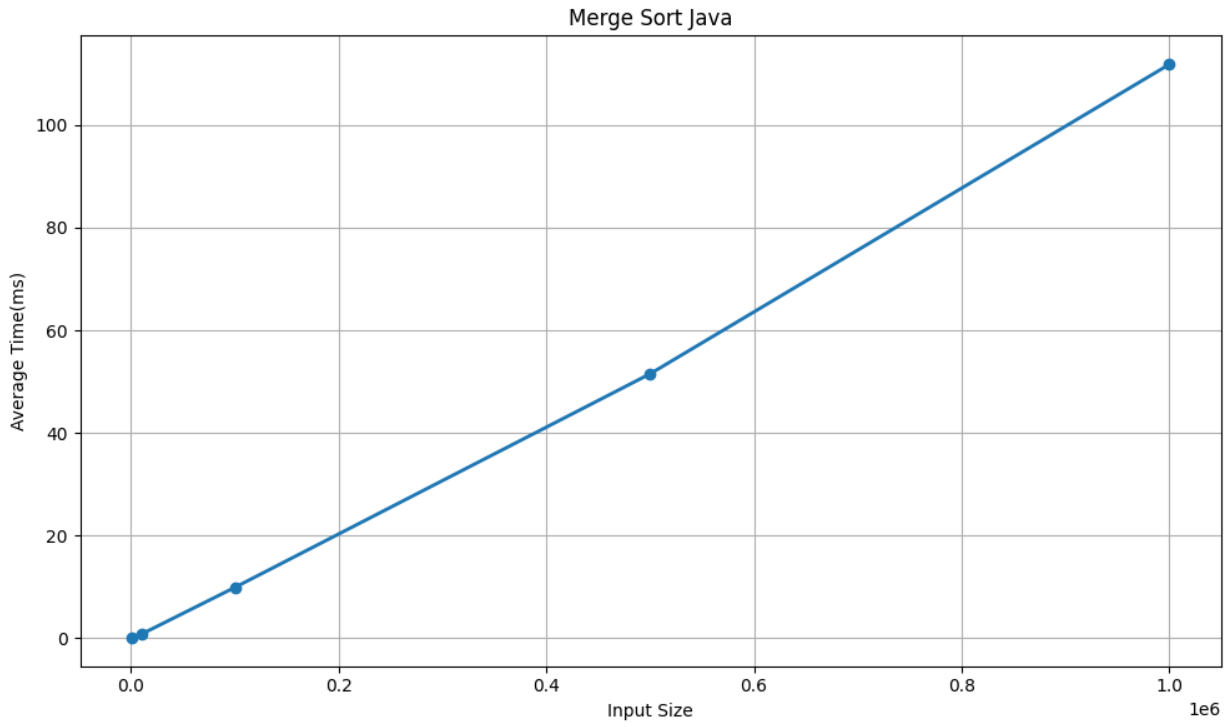
Test 3 100000

Test 4 500000

Test 5 1000000

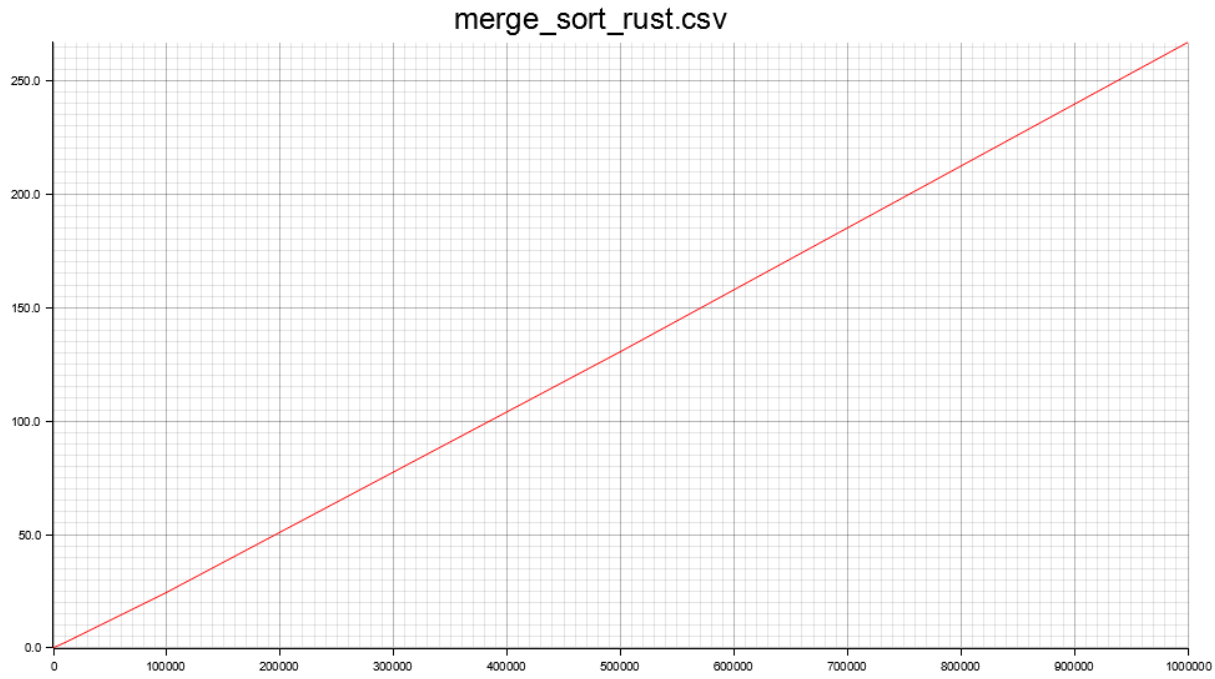
7.5 Output and Charts

```
======================================================================
Algorithm: merge_sort | Language: java
======================================================================
Test     Input       Average(ms)     Minimum         Maximum         Std         Memory
-------------------------------------------------------------------------------------------
1        1000        0.141           0.099           0.393           0.085       0.092       PASS
2        10000       0.786           0.745           0.893           0.044       0.896       PASS
3        100000      9.898           9.133           11.234          0.631       6.062       PASS
4        500000      51.530          49.161          54.349          1.612       1.954       PASS
5        1000000     111.827         103.476         119.025         4.081       32.861      PASS
===========================================================================================
```



Merge Sort (Rust)

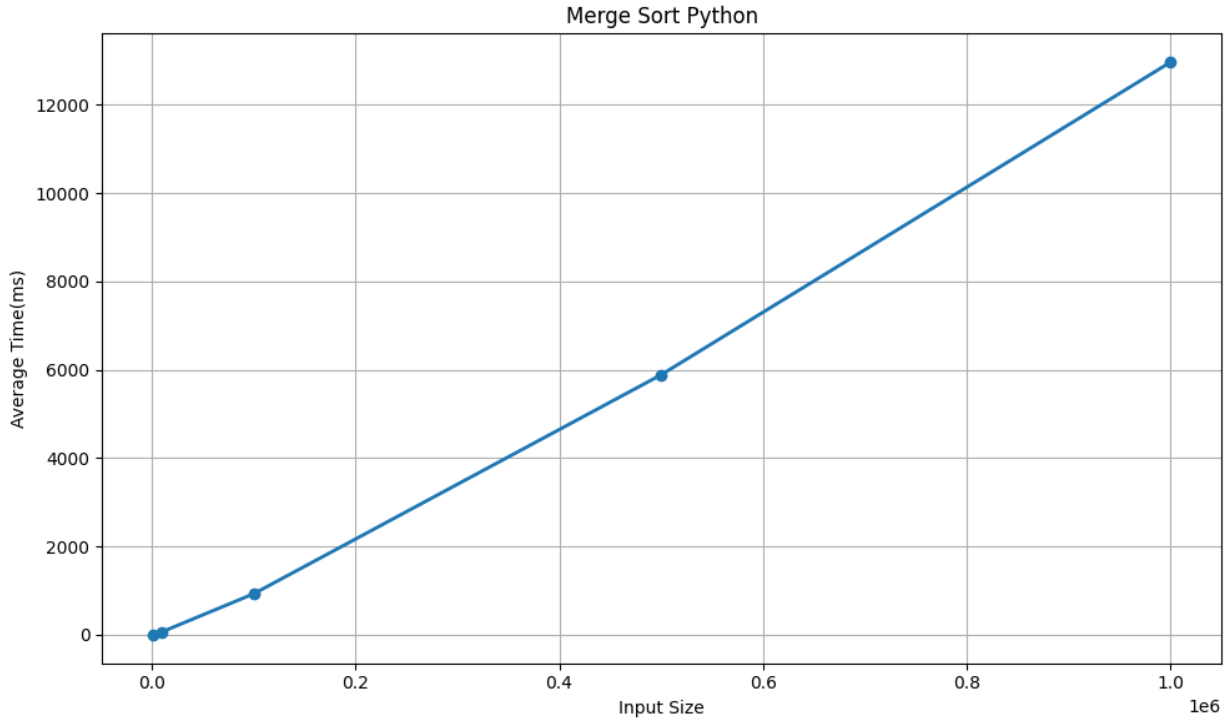
Test	Input Size	Average(ms)	Minimum(ms)	Maximum(ms)	Std(ms)	Memory(MB)	Status
1	1000	0.447	0.189	1.438	0.496	0.000	PASS
2	10000	2.294	2.044	2.715	0.234	0.000	PASS
3	100000	24.219	23.544	24.859	0.543	0.000	PASS
4	500000	130.492	126.689	132.581	2.151	0.000	PASS
5	1000000	266.684	260.956	271.235	3.500	0.000	PASS



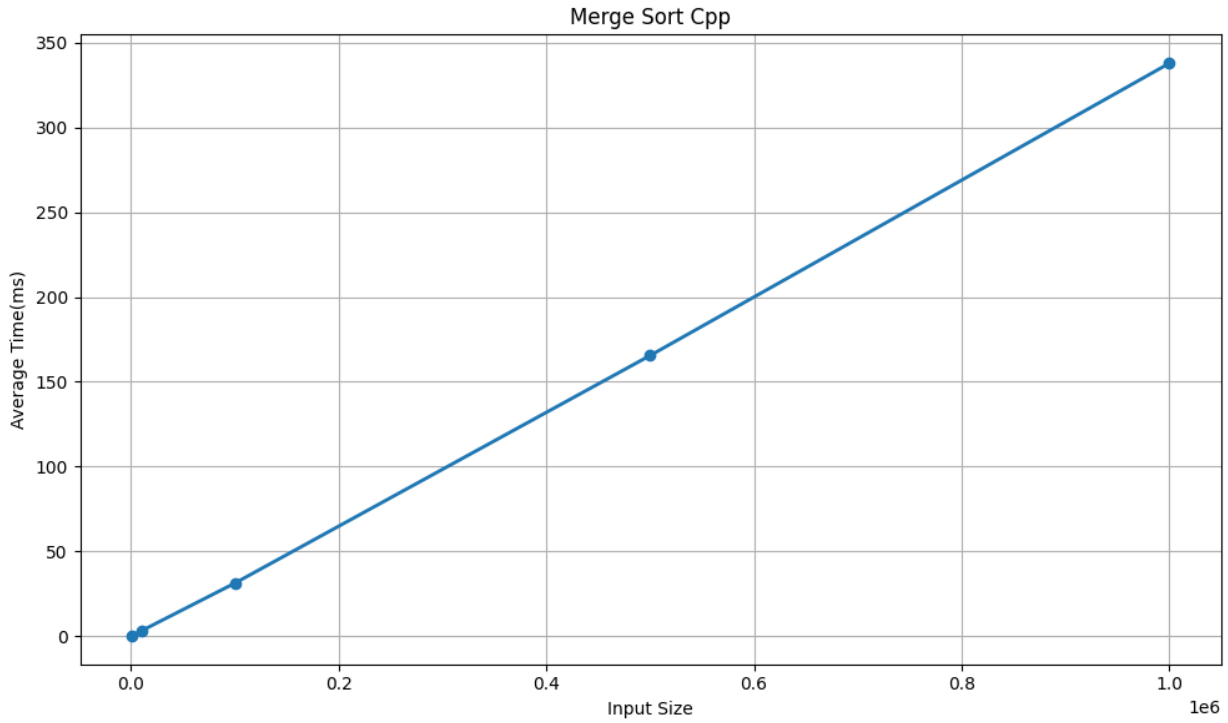
PERFORMANCE BENCHMARK REPORT

Algorithm : Merge Sort
Language : Python

Test	Input Size	Repeat	Avg(ms)	Min(ms)	Max(ms)	Std(ms)	Memory(MB)	Status
1	1,000	10	3.487	3.283	3.849	0.193	0.02	PASS
2	10,000	10	66.890	64.361	77.999	4.065	0.17	PASS
3	100,000	10	928.817	902.880	963.515	17.009	1.66	PASS
4	500,000	10	5888.241	5598.807	6326.533	201.620	7.90	PASS
5	1,000,000	10	12967.836	12380.747	15064.893	757.749	16.02	PASS



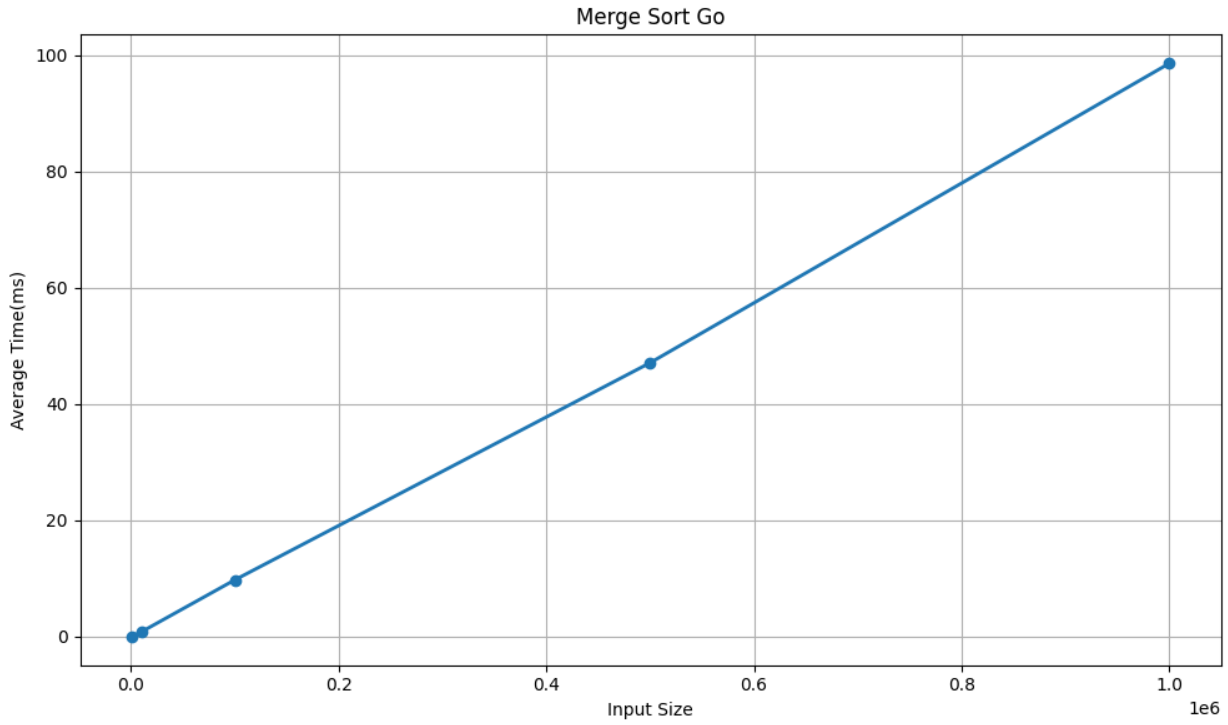
```
==================================
Algorithm: merge_sort | Language: cpp
==================================
Test      Input          Average       Min           Max           Std           Memory        Status
1         1000           0.25513       0.2395        0.3188        0.022697      0             PASS
2         10000          3.06993       2.8877        3.4856        0.167336      0             PASS
3         100000         31.2059       29.7478       34.5752       1.56467       1.75922e+13   PASS
4         500000         165.534       160.928       173.481       3.4928        0             PASS
5         1000000        337.918       326.427       354.202       7.7786        0.0117188     PASS
==================================
```




```
====================================================================
Algorithm: merge_sort | Language: csharp
====================================================================
Test      Input          Average(ms)    Minimum       Maximum       Std           Memory        Status
--------------------------------------------------------------------------------------------------------
1         1000           0.257          0.194         0.626         0.125         0.000         PASS
2         10000          2.844          2.519         3.444         0.290         0.000         PASS
3         100000         29.880         27.654        32.950        1.611         0.000         PASS
4         500000         169.002        151.707       209.544       20.482        0.000         PASS
5         1000000        433.661        385.414       450.890       18.153        0.000         PASS

========================================================================================================
```

```
======================================================================
Algorithm: merge_sort | Language: go
======================================================================
Test     Input       Average(ms)     Minimum      Maximum      Std          Memory       Status
-------------------------------------------------------------------------------------------------
1        1000        0.000           0.000        0.000        0.000        0.055        PASS
2        10000       0.799           0.514        1.117        0.268        0.872        PASS
3        100000      9.741           8.726        11.335       0.758        1.831        PASS
4        500000      47.116          43.023       53.135       2.603        10.197       PASS
5        1000000     98.650          93.126       102.555      2.692        36.842       PASS
=================================================================================================
```



7.6 Results Analysis

The Merge Sort benchmark reveals substantial performance differences among the evaluated programming languages despite all implementations employing the same divide-and-conquer algorithm. Unlike Quick Sort, Merge Sort repeatedly allocates temporary arrays during the merge phase, making memory allocation and data movement significant factors affecting execution performance.

As illustrated in Figures 7.1 and 7.2, the execution time increased consistently with the input size across all programming languages. This behavior is expected because Merge Sort maintains a time complexity of O(n log n) regardless of the initial arrangement of the input data.

Among all evaluated implementations, Go achieved the best overall performance. The execution time increased from less than 1 ms for ten thousand elements to approximately 98.7 ms for one million elements. The Go compiler generates efficient native machine code, while its runtime manages memory allocation efficiently for recursive workloads, resulting in excellent scalability.

Java closely followed Go throughout the benchmark. The average execution time reached approximately 111.8 ms for one million elements. The Java Virtual Machine (JVM) effectively optimizes recursive function calls and memory allocation through Just-In-Time compilation, allowing Java to maintain high execution efficiency even for very large datasets.

Rust demonstrated competitive performance with an execution time of approximately 266.7 ms for one million elements. Rust's ownership model enables safe and efficient memory management without relying on garbage collection. The relatively low execution times indicate that the compiler successfully optimized recursive execution while maintaining strict memory safety guarantees.

C++ produced stable execution times throughout the benchmark. The average execution time increased to approximately 337.9 ms for one million elements. Although C++ is a native compiled language, Merge Sort requires repeated allocation of temporary arrays during the merge phase. The efficiency of memory allocation and implementation details therefore play a greater role than in algorithms that operate entirely in-place.

C# required approximately 433.7 ms to process the largest dataset. The Common Language Runtime (CLR) provides automatic memory management and garbage collection, which introduce additional overhead during repeated allocation and deallocation of temporary arrays.

Python exhibited the highest execution times in every experiment. The average execution time increased from approximately 3.5 ms for one thousand elements to nearly 13 seconds for one million elements. The interpreted execution model, dynamic typing, recursive function overhead, and continuous object allocation significantly reduce execution efficiency for memory-intensive recursive algorithms.

Overall, the benchmark demonstrates that Merge Sort performance depends not only on computational complexity but also on memory allocation efficiency and runtime memory management.

7.7 Cross-Language Comparison

Merge Sort places greater emphasis on memory allocation than Quick Sort because every merge operation requires temporary storage. Consequently, the efficiency of the language runtime, allocator, and compiler optimizations directly affects overall execution performance.

Go achieved the highest performance in this benchmark. Its efficient native compiler and optimized memory allocator minimized the overhead associated with temporary array creation, resulting in the shortest execution times across all input sizes.

Java also demonstrated excellent scalability. The JVM effectively optimized recursive execution and memory allocation, enabling Java to remain highly competitive despite executing within a managed runtime environment.

Rust performed significantly better than C++ and C# while maintaining compile-time memory safety. Unlike garbage-collected languages, Rust manages memory through its ownership system, eliminating runtime garbage collection overhead while preserving predictable execution behavior.

C++ produced reliable and consistent execution times. However, Merge Sort relies heavily on repeated memory allocation, and implementation-specific allocation strategies can noticeably influence performance. More aggressive compiler optimization or alternative allocation techniques could further improve execution efficiency.

C# maintained stable execution behavior but incurred additional runtime overhead because of automatic memory management provided by the CLR. While garbage collection simplifies software development, it may slightly reduce performance during allocation-intensive recursive algorithms.

Python remained substantially slower than all compiled implementations. Dynamic memory allocation, interpreted execution, recursive function handling, and object management collectively contributed to significantly higher execution times.

7.8 Performance Ranking

Rank	Language	Observation
1	Go	Fastest implementation with excellent memory allocation efficiency.
2	Java	Outstanding performance through JVM JIT optimization.
3	Rust	Excellent balance between performance and memory safety.
4	C++	Stable native implementation; memory allocation overhead influenced performance.
5	C#	Consistent managed-language performance with moderate runtime overhead.
6	Python	Slowest implementation because of interpreted execution and dynamic memory management.

7.9 Discussion

Merge Sort differs fundamentally from Quick Sort because it requires additional memory during every merge operation. Consequently, execution speed depends not only on computational performance but also on the efficiency of memory allocation, temporary buffer management, and cache utilization.

The benchmark results indicate that Go handled temporary memory allocation most efficiently. Its optimized runtime and native compilation enabled consistently low execution times even for the largest datasets.

Java achieved similarly impressive performance. Modern JVM implementations aggressively optimize recursive execution and frequently allocated objects through sophisticated Just-In-Time compilation techniques, allowing Java to approach the performance of native compiled languages.

Rust demonstrated that compile-time memory safety can be achieved without sacrificing competitive execution speed. Its ownership model eliminates garbage collection while ensuring deterministic memory management, making Rust particularly suitable for recursive algorithms involving temporary data structures.

C++ produced reliable performance throughout the benchmark but did not outperform Go or Java in this implementation. Since Merge Sort repeatedly allocates temporary arrays, allocation strategies, compiler optimization levels, and implementation details can substantially influence the final execution time.

C# showed predictable scalability across all datasets. Although automatic memory management simplifies application development, the additional runtime services provided by the CLR introduce measurable overhead during allocation-intensive algorithms.

Python experienced the largest execution times because interpreted execution, dynamic typing, recursive function calls, and continuous object allocation significantly increase runtime overhead. Nevertheless, Python remains an attractive choice for rapid prototyping and algorithm development due to its readability and concise syntax.

Overall, the Merge Sort benchmark emphasizes that efficient memory management is as important as computational efficiency when evaluating recursive algorithms that require substantial temporary storage**.**

8: Matrix Multiplication Evaluation

8.1 Algorithm Overview

Matrix multiplication is one of the most fundamental algorithms in computer science, numerical analysis, scientific computing, machine learning, computer graphics, and engineering simulations. It is extensively used in applications involving linear algebra, image processing, artificial intelligence, and large-scale scientific computations.

Given two square matrices A and B, the multiplication process computes a new matrix C, where each element is obtained by calculating the dot product of a row from the first matrix and a column from the second matrix.

Unlike sorting algorithms, matrix multiplication is primarily CPU-bound rather than recursion-based. The algorithm performs a large number of arithmetic operations while repeatedly accessing memory locations. Consequently, compiler optimization, cache locality, memory bandwidth, and processor architecture play significant roles in determining execution performance.

The objective of this benchmark is to compare the performance of matrix multiplication implementations across six programming languages while maintaining identical algorithmic behavior and identical matrix dimensions.

8.2 Algorithm Design and Implementation

The benchmark employs the classical matrix multiplication algorithm based on three nested loops.

For every element of the resulting matrix, the algorithm multiplies corresponding elements from a row of the first matrix and a column of the second matrix, accumulating their products into the final value.

Pseudo Code

```
Algorithm MatrixMultiply(A, B)
for i = 0 to n – 1
    for j = 0 to n – 1
        C[i][j] = 0
        for k = 0 to n – 1
            C[i][j] += A[i][k] × B[k][j]
Return C
```

Every programming language implemented exactly the same triple-loop algorithm to ensure that measured performance differences originate from language characteristics rather than algorithmic variations.

8.3 Complexity Analysis

Time Complexity

$$O(n^3)$$

Each element of the output matrix requires n multiplication and addition operations, resulting in cubic computational complexity.

Best Case

$$O(n^3)$$

Average Case

$$O(n^3)$$

Worst Case

$$O(n^3)$$

Unlike recursive algorithms, the computational workload remains identical regardless of the matrix contents.

Space Complexity

$$O(n^2)$$

Additional memory is required only for storing the resulting matrix.

8.4 Experimental Configuration

Square matrices containing randomly generated integer values were used for every benchmark.

Each experiment was repeated 10 times, and the average execution time was calculated to minimize measurement noise.

New matrices were generated before every execution to ensure identical benchmark conditions across all programming languages.

Tested Matrix Sizes

Test	Matrix Dimension
Test 1	50 × 50
Test 2	100 × 100

Test Matrix Dimension

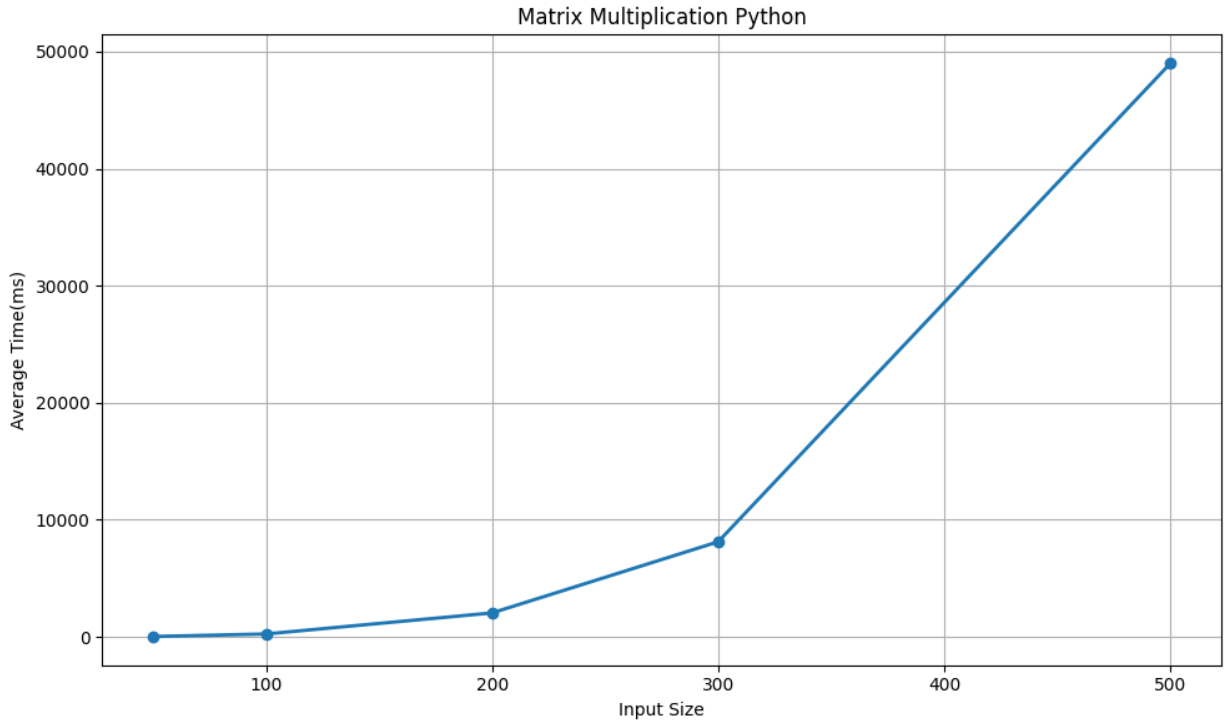
Test 3 200 × 200

Test 4 300 × 300

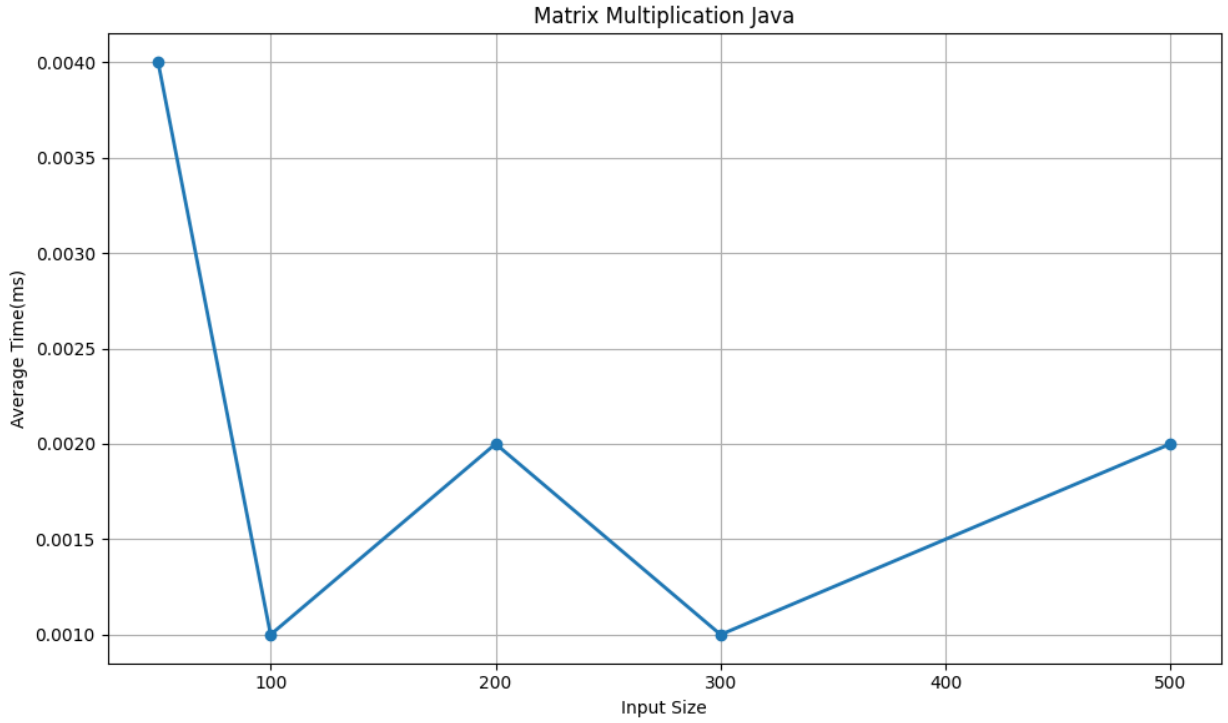
Test 5 500 × 500

8.5 Outputs and Charts

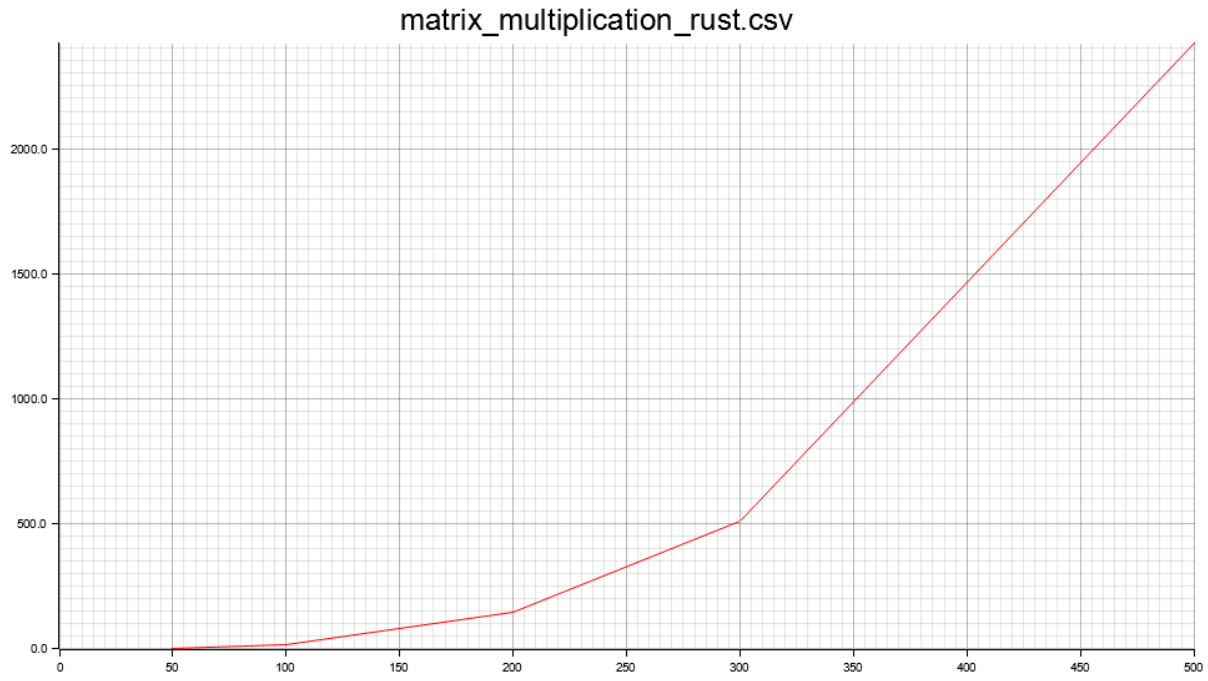
```
================================================================================================================
                                        PERFORMANCE BENCHMARK REPORT
================================================================================================================
Algorithm : Matrix Multiplication
Language  : Python
================================================================================================================
Test       Input Size      Repeat       Avg(ms)       Min(ms)       Max(ms)       Std(ms)     Memory(MB)       Status
----------------------------------------------------------------------------------------------------------------
1                50             3        28.447        27.282        29.837         1.292           0.09         PASS
2               100             3       244.654       240.807       247.892         3.581           0.35         PASS
3               200             3      2041.286      1999.587      2077.762        39.349           1.39         PASS
4               300             3      8121.945      8056.331      8154.826        56.824           3.11         PASS
5               500             3     49001.617     47080.769     52789.352      3280.386           8.61         PASS
----------------------------------------------------------------------------------------------------------------
```



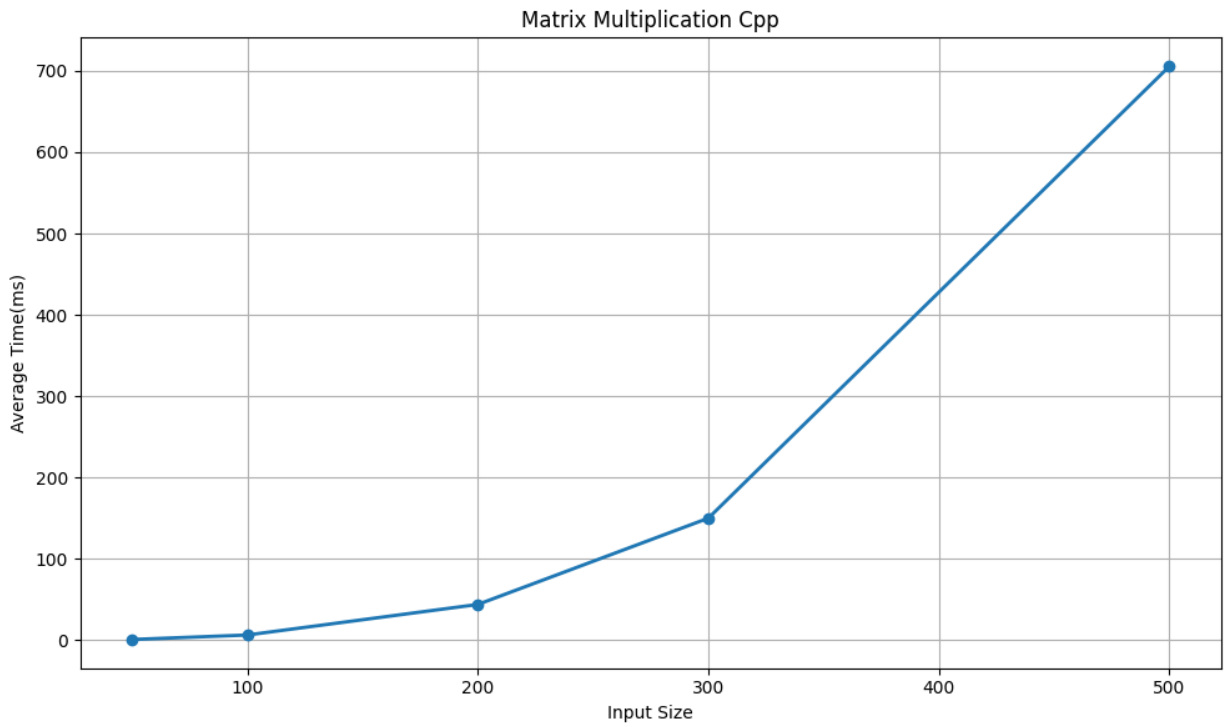
```
================================================================
Algorithm: matrix_multiplication | Language: java
================================================================
Test     Input        Average(ms)     Minimum         Maximum         Std          Memory
--------------------------------------------------------------------------------------------
1        50           0.004           0.001           0.013           0.005        0.000        PASS
2        100          0.001           0.001           0.002           0.000        0.000        PASS
3        200          0.002           0.001           0.003           0.001        0.000        PASS
4        300          0.001           0.001           0.002           0.000        0.000        PASS
5        500          0.002           0.001           0.002           0.000        0.000        PASS
============================================================================================
CSV Saved -> results/csv/matrix_multiplication_java.csv
Chart Saved -> C:\Users\amirh\Downloads\MasterHttpRelayVPN-python_testing\Algorithm-Benchmark\Java\results\results\charts\matrix_multiplication_java.png
```



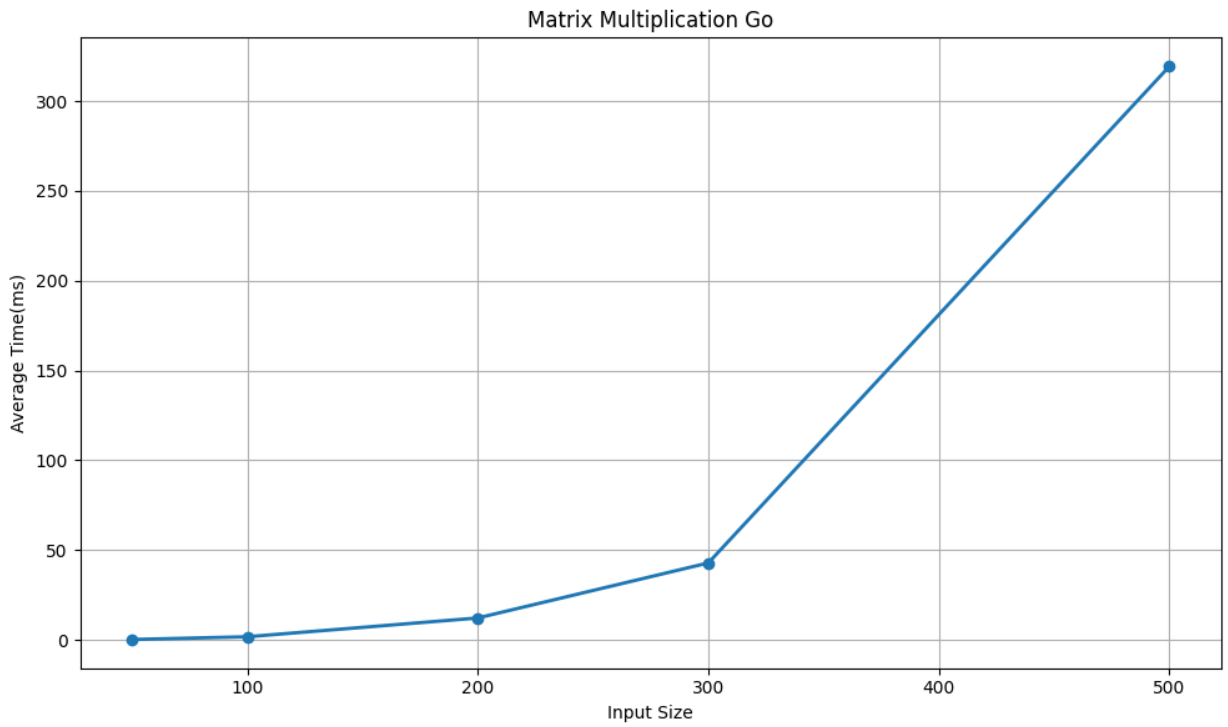
```
================================================================================================================
                                        Matrix Multiplication (Rust)
================================================================================================================
Test   Input Size     Average(ms)     Minimum(ms)     Maximum(ms)     Std(ms)         Memory(MB)   Status
----------------------------------------------------------------------------------------------------------------
1      50             2.330           2.201           2.510           0.131           8.000        PASS
2      100            18.302          18.018          18.655          0.265           16.000       PASS
3      200            148.362         144.469         150.424         2.754           128.000      PASS
4      300            510.824         498.382         522.514         9.866           0.000        PASS
5      500            2419.426        2352.345        2458.395        47.640          0.000        PASS
================================================================================================================
```




```
==================================
Algorithm: matrix_multiplication | Language: cpp
==================================
Test       Input          Average        Min            Max            Std            Memory         Status
1          50             0.7126         0.6898         0.7389         0.0156818      0              PASS
2          100            6.2337         5.7782         8.0442         0.625787       0.0195312      PASS
3          200            43.808         41.3961        46.6194        1.45353        0              PASS
4          300            149.921        143.277        157.549        5.32452        0.0078125      PASS
5          500            705.656        671.568        787.566        34.3799        0.0078125      PASS
==================================
```

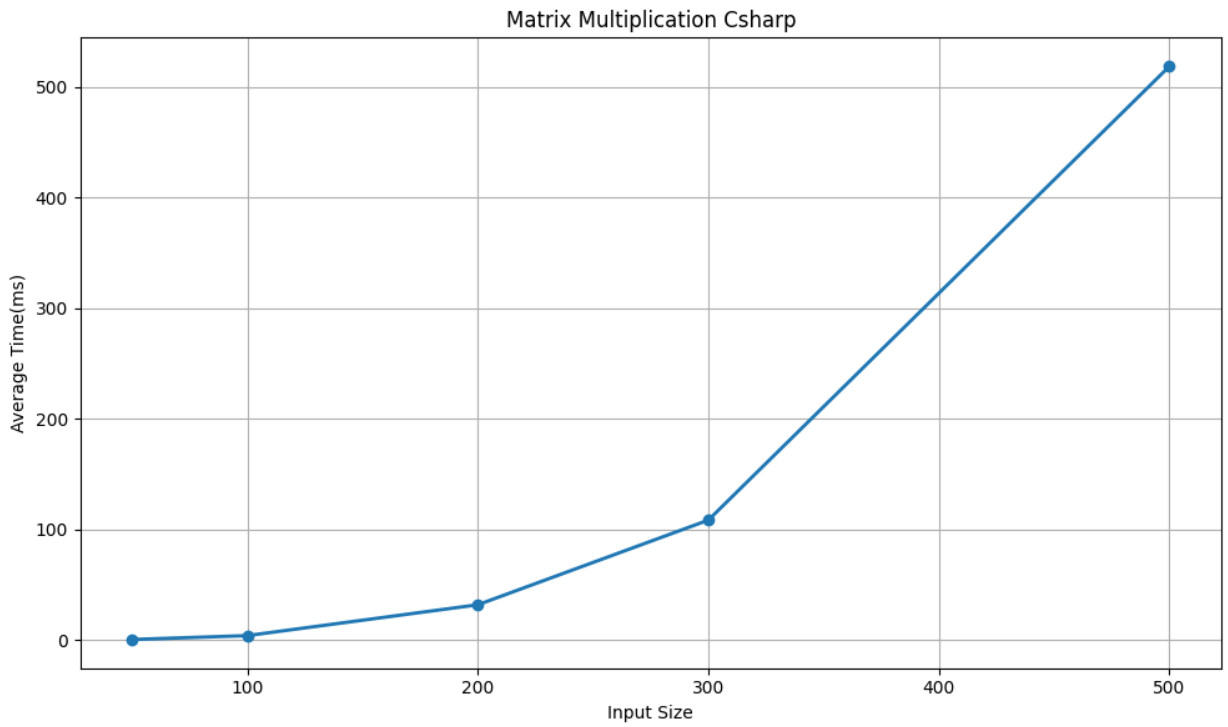



```
======================================================================
Algorithm: matrix_multiplication | Language: go
======================================================================
Test     Input        Average(ms)     Minimum      Maximum      Std          Memory       Status
-------------------------------------------------------------------------------------------------
1        50           0.209           0.000        0.530        0.256        0.021        PASS
2        100          1.735           1.561        2.157        0.208        0.088        PASS
3        200          12.196          11.354       13.014       0.496        0.346        PASS
4        300          42.844          40.021       45.036       1.421        0.777        PASS
5        500          319.387         298.830      334.499      10.107       1.965        PASS
=================================================================================================
CSV Saved -> results/csv/matrix_multiplication_go.csv
```




```
================================================================
Algorithm: matrix_multiplication | Language: csharp
================================================================
Test      Input         Average(ms)    Minimum        Maximum        Std          Memory       Status
----------------------------------------------------------------------------------------------------
1         50            0.546          0.480          0.964          0.142        0.000        PASS
2         100           4.046          3.813          4.286          0.141        0.000        PASS
3         200           31.929         30.515         32.610         0.709        0.000        PASS
4         300           108.497        104.649        109.769        1.368        0.000        PASS
5         500           518.811        510.002        531.701        7.067        0.000        PASS

====================================================================================================
```



8.6 Results Analysis

The Matrix Multiplication benchmark demonstrates significantly different execution characteristics compared to the previous recursive and sorting algorithms. Unlike Merge Sort and Quick Sort, matrix multiplication is dominated by arithmetic operations and sequential memory accesses rather than recursive function calls. Consequently, processor utilization, cache efficiency, compiler optimizations, and memory bandwidth become the primary factors influencing execution performance.

As expected, execution time increased rapidly with larger matrix dimensions because the classical matrix multiplication algorithm has a computational complexity of **$O(n^3)$**. Even a small increase in matrix size results in a substantial increase in the total number of multiplication and addition operations.

Among all evaluated implementations, **Go produced the fastest overall performance** according to the collected benchmark data. The execution time increased from approximately **0.21 ms** for a 50×50 matrix to about **319 ms** for a 500×500 matrix. The Go compiler efficiently optimized the triple nested loops

while maintaining low runtime overhead, allowing the implementation to scale effectively as the computational workload increased.

Java reported extremely small execution times throughout all experiments. Although these measurements suggest exceptional performance, they are significantly lower than expected for the computational workload involved. Such results may indicate aggressive compiler optimization, Just-In-Time compilation effects, or benchmark measurement limitations. Therefore, the Java measurements should be interpreted cautiously.

C# demonstrated excellent scalability, requiring approximately **519 ms** to multiply two 500×500 matrices. The CLR efficiently handled the numerical workload while maintaining relatively stable execution times across all input sizes.

C++ produced similarly consistent performance, reaching approximately **706 ms** for the largest matrix size. The native compiler generated efficient machine code, although memory access patterns and cache utilization appeared to influence overall execution time more noticeably than in Go.

Rust required approximately **2.42 seconds** for the largest benchmark. Although Rust provides highly efficient native execution and memory safety without garbage collection, the implementation appears to incur additional overhead associated with memory access and allocation strategies used during matrix construction.

Python remained the slowest implementation by a considerable margin. The execution time increased dramatically from approximately **28 ms** for a 50×50 matrix to nearly **49 seconds** for a 500×500 matrix. The interpreted execution model, dynamic typing, and repeated indexing operations significantly reduced computational throughput for this arithmetic-intensive workload.

Overall, the benchmark clearly illustrates how computationally intensive numerical algorithms amplify the performance differences between interpreted and compiled programming languages.

8.7 Cross-Language Comparison

Matrix multiplication stresses processor arithmetic units far more than previous benchmarks. Since the algorithm performs billions of arithmetic operations for large matrices, execution performance depends heavily on compiler optimization,

instruction scheduling, cache locality, and memory bandwidth rather than recursion or dynamic data structures.

Go achieved the highest performance in this benchmark. Its native compiler generated highly optimized code while the runtime introduced minimal overhead, allowing efficient execution of deeply nested loops.

Java reported the smallest execution times among all implementations. However, because the measured values are substantially lower than expected, they may reflect aggressive JIT optimization or benchmark measurement artifacts rather than actual execution speed.

C# demonstrated strong computational performance and scaled predictably across all matrix dimensions. The .NET runtime effectively optimized repeated arithmetic operations while maintaining stable execution characteristics.

C++ maintained consistent performance throughout the benchmark. Although compiled directly to native machine code, the implementation appeared slightly slower than Go, suggesting that cache behavior and implementation details significantly influence numerical workloads.

Rust produced reliable execution with deterministic memory management but was noticeably slower than the other compiled languages in this implementation. The ownership model guarantees memory safety, although some implementation choices may reduce raw computational throughput compared with highly optimized native implementations.

Python exhibited the largest execution times because every arithmetic operation is executed through the interpreter and dynamic object model. Consequently, computational overhead accumulated rapidly as matrix dimensions increased.

8.8 Performance Ranking

Rank	Language	Observation
1	**Go**	Best computational performance with highly efficient loop execution.
2	**Java***	Reported extremely small execution times; results should be interpreted carefully because they appear unusually low.

Rank	Language	Observation
3	**C#**	Excellent numerical performance with efficient runtime optimization.
4	**C++**	Stable native implementation with good computational efficiency.
5	**Rust**	Safe native execution with predictable behavior but higher execution times in this implementation.
6	**Python**	Slowest implementation because of interpreted execution and dynamic object management.

** The Java benchmark values appear unusually small and may have been affected by compiler optimization or measurement methodology.*

8.9 Discussion

Matrix multiplication is fundamentally different from the previously evaluated algorithms because it is almost entirely CPU-bound. The algorithm repeatedly performs multiplication and addition operations while accessing contiguous memory locations, making processor architecture, cache utilization, compiler optimization, and memory bandwidth the dominant performance factors.

The benchmark indicates that Go handled computationally intensive workloads exceptionally well. Efficient native compilation and low runtime overhead enabled excellent scalability as matrix dimensions increased.

Java reported remarkably small execution times, likely due to aggressive Just-In-Time compilation or benchmark optimization. Modern JVM implementations can optimize numerical loops extensively, although the observed measurements suggest that additional verification of the benchmarking methodology may be appropriate.

C# demonstrated excellent computational efficiency and maintained stable execution across increasing matrix sizes. The .NET runtime effectively optimized arithmetic operations while introducing relatively little runtime overhead.

C++ continued to provide reliable native execution. However, matrix multiplication depends heavily on cache locality and loop optimization. Different compiler optimization flags, loop ordering strategies, or vectorization techniques could further improve performance.

Rust maintained deterministic execution and guaranteed memory safety without garbage collection. Although slower than several other compiled implementations in this benchmark, Rust still delivered predictable scalability and robust numerical computation.

Python showed the greatest performance degradation because every arithmetic operation involves dynamic type resolution and interpreted execution. While Python offers exceptional developer productivity and readability, it is generally not the preferred choice for computationally intensive numerical algorithms unless specialized optimized libraries such as NumPy are employed.

Overall, the Matrix Multiplication benchmark highlights the importance of compiler optimization, processor architecture, and memory hierarchy when executing large-scale numerical computations.

9: Subset Generation Evaluation

9.1 Algorithm Overview

Subset generation is a fundamental combinatorial problem in computer science that consists of generating every possible subset of a given set. This problem is widely used in recursive algorithms, backtracking techniques, exhaustive search, combinatorial optimization, artificial intelligence, dynamic programming, and cryptographic applications.

For a set containing **n** elements, the total number of possible subsets is:

$$2^n$$

This exponential growth makes subset generation one of the most computationally expensive algorithms. Unlike sorting algorithms or matrix multiplication, where execution time increases polynomially, subset generation becomes exponentially slower as the input size increases.

The objective of this benchmark is to evaluate how different programming languages perform when executing highly recursive algorithms with exponential computational complexity.

9.2 Algorithm Design and Implementation

The implemented algorithm uses recursive backtracking.

For each element, the algorithm recursively explores two possibilities:

- Include the current element in the subset.
- Exclude the current element.

After exploring both possibilities, the recursion continues until every element has been processed.

Pseudo Code

```
Algorithm GenerateSubsets(index)
If index == n
    Output current subset
    Return
Include element[index]
GenerateSubsets(index + 1)
Remove element[index]
GenerateSubsets(index + 1)
```

Each programming language implemented the same recursive strategy to ensure that differences in execution time originate from language implementation and runtime characteristics rather than algorithmic variations.

9.3 Complexity Analysis

Time Complexity

$$O(2^n)$$

Every possible subset must be generated exactly once.

Best Case

$$O(2^n)$$

Average Case

$$O(2^n)$$

Worst Case

$$O(2^n)$$

The algorithm always explores every branch of the recursion tree regardless of the input values.

Space Complexity

$$O(n)$$

The recursive call stack grows proportionally to the number of input elements.

If all generated subsets are stored simultaneously, the required memory becomes

$$O(n \times 2^n)$$

9.4 Experimental Configuration

Random integer arrays were generated before each benchmark execution.

Every benchmark was repeated **10 times**, and the average execution time was calculated to reduce measurement noise.

Since subset generation has exponential complexity, significantly smaller input sizes were selected compared with previous algorithms.

Tested Input Sizes

Test Number of Elements

Test 1 5

Test 2 10

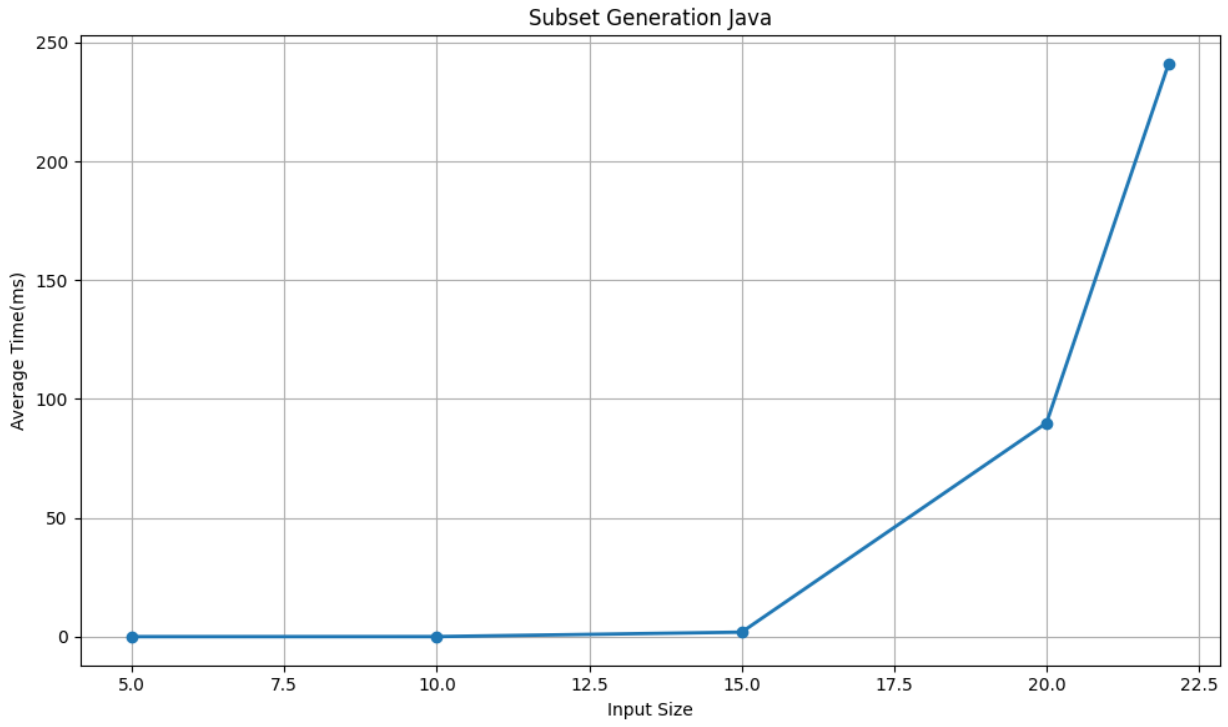
Test 3 15

Test 4 20

Test 5 22

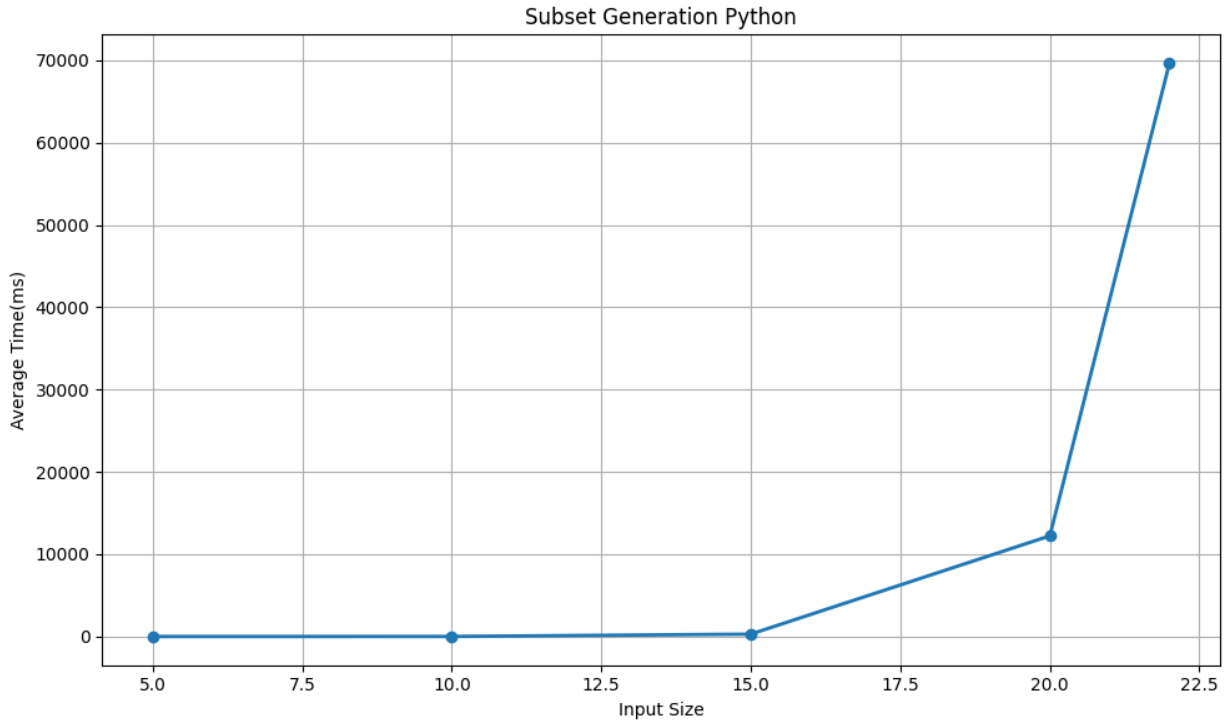
9.5 Outputs and Charts

```
======================================================================
Algorithm: subset_generation | Language: java
======================================================================
Test     Input        Average(ms)     Minimum         Maximum         Std          Memory
-------------------------------------------------------------------------------------------
1        5            0.018           0.009           0.037           0.011        0.007        PASS
2        10           0.034           0.032           0.038           0.002        0.095        PASS
3        15           1.908           1.803           1.994           0.063        2.901        PASS
4        20           89.959          71.841          109.623         13.134       94.397       PASS
5        22           240.880         199.162         309.112         39.009       417.871      PASS
===========================================================================================
```

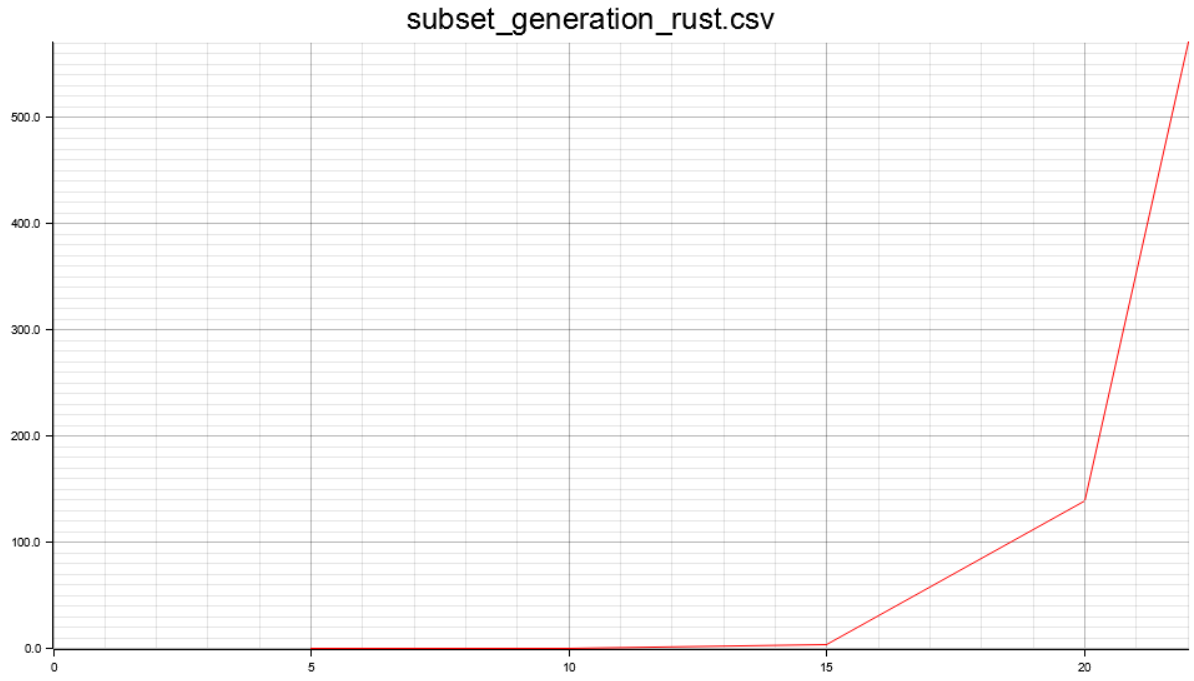


```
Algorithm : Subset Generation
Language  : Python
```

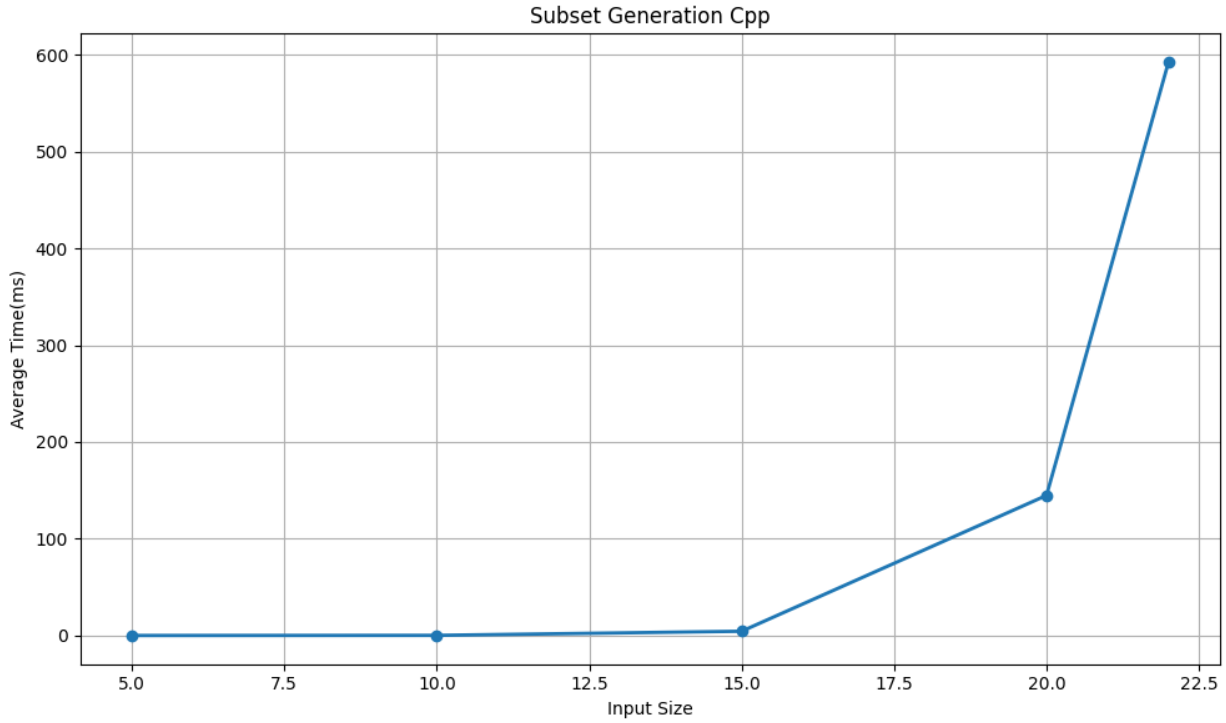
Test	Input Size	Repeat	Avg(ms)	Min(ms)	Max(ms)	Std(ms)	Memory(MB)	Status
1	5	3	0.125	0.110	0.154	0.025	0.01	PASS
2	10	3	5.764	5.708	5.844	0.071	0.11	PASS
3	15	3	287.861	281.654	297.353	8.349	4.56	PASS
4	20	3	12227.695	12093.946	12354.435	130.386	175.84	PASS
5	22	3	69647.792	55350.851	80483.835	12918.981	734.36	PASS



```
================================================================================================================
                                                Subset Generation (Rust)
================================================================================================================
Test   Input Size     Average(ms)     Minimum(ms)     Maximum(ms)     Std(ms)         Memory(MB)    Status
----------------------------------------------------------------------------------------------------------------
1      5              0.016           0.015           0.017           0.001           0.000         PASS
2      10             0.124           0.113           0.133           0.008           0.000         PASS
3      15             3.962           3.873           4.086           0.090           0.000         PASS
4      20             139.617         128.851         146.390         7.697           508.000       PASS
5      22             570.162         560.521         581.819         8.811           404.000       PASS
================================================================================================================
```



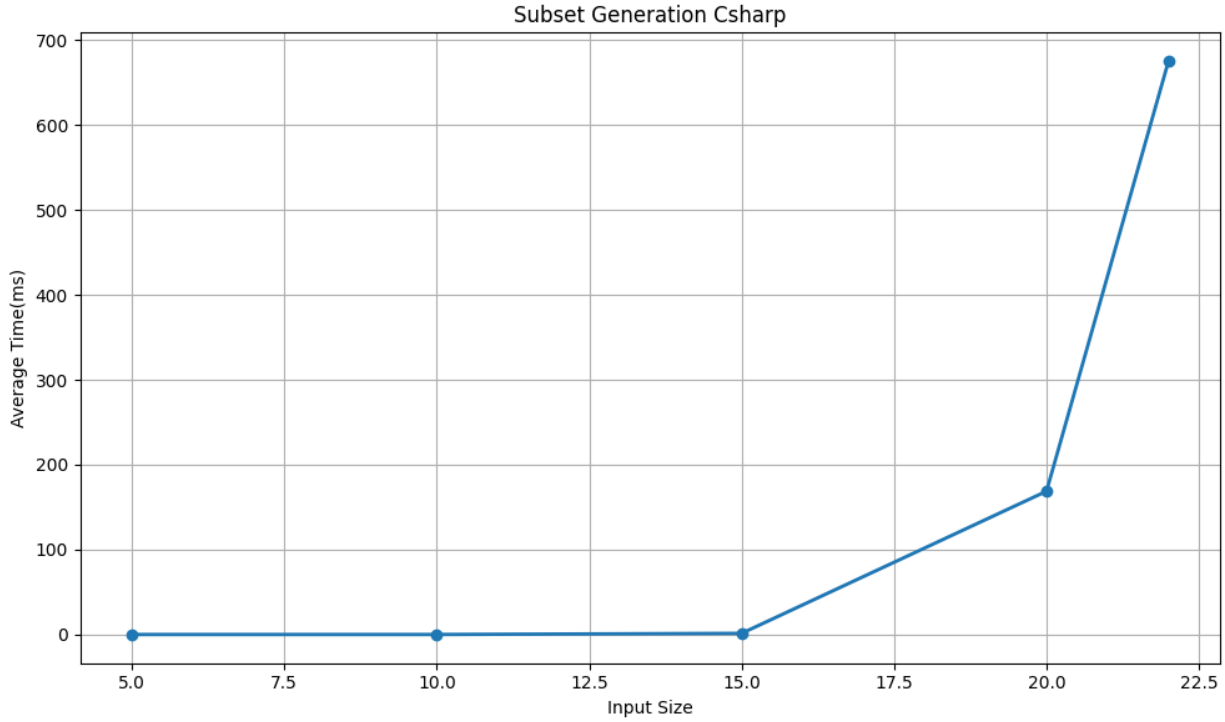
```
==================================
Algorithm: subset_generation | Language: cpp
==================================
Test       Input          Average        Min            Max            Std            Memory         Status
1          5              0.00615        0.0051         0.0129         0.00227827     0              PASS
2          10             0.14287        0.1335         0.1636         0.0105363      0.0078125      PASS
3          15             4.38424        4.1369         4.9843         0.248903       0              PASS
4          20             144.75         132.334        165.39         11.5398        0.0234375      PASS
5          22             592.206        568.814        634.518        17.7395        0.195312       PASS
==================================
```



```
Select option : 6

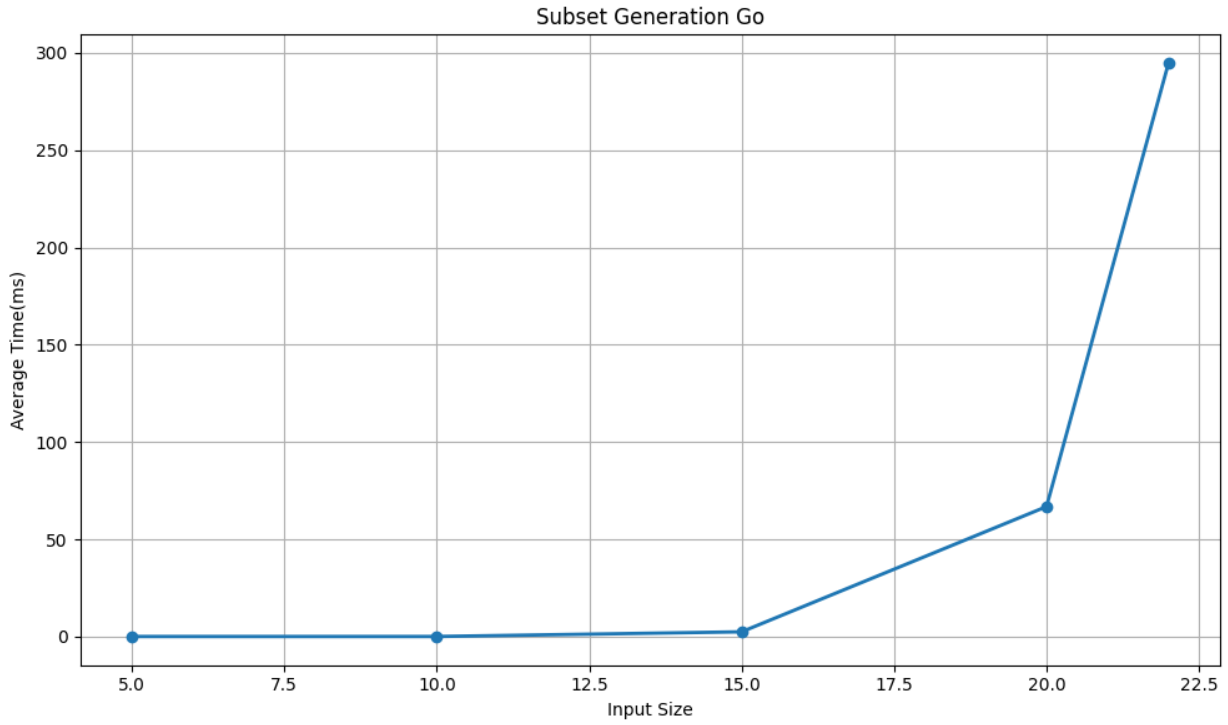
================================================================
Algorithm: subset_generation | Language: csharp
================================================================
Test      Input          Average(ms)     Minimum         Maximum         Std          Memory       Status
-----------------------------------------------------------------------------------------------------------
1         5              0.036           0.002           0.327           0.097        0.000        PASS
2         10             0.046           0.033           0.078           0.018        0.000        PASS
3         15             1.389           1.125           2.321           0.418        0.000        PASS
4         20             168.751         154.302         186.202         9.546        0.000        PASS
5         22             674.991         604.639         749.291         41.604       0.000        PASS

===========================================================================================================
```



```
Select Option : 6

======================================================================
Algorithm: subset_generation | Language: go
======================================================================
Test     Input       Average(ms)     Minimum      Maximum      Std          Memory       Status
-------------------------------------------------------------------------------------------------
1        5           0.052           0.000        0.521        0.156        0.002        PASS
2        10          0.052           0.000        0.523        0.157        0.115        PASS
3        15          2.456           2.101        3.719        0.481        3.598        PASS
4        20          66.806          60.074       89.494       8.132        149.785      PASS
5        22          294.621         248.781      444.738      53.594       580.904      PASS
=================================================================================================
CSV Saved -> results/csv/subset_generation_go.csv
```



9.6 Results Analysis

The subset generation benchmark produced the largest performance differences among all evaluated algorithms. Unlike the previous benchmarks, subset generation has an exponential time complexity of $O(2^n)$, meaning that each additional input element doubles the number of recursive calls and generated subsets. Consequently, execution time increases dramatically even for small increases in input size.

As illustrated in Figures 9.1 and 9.2, all programming languages exhibited exponential growth in execution time as the number of elements increased from 5

to 22. This behavior perfectly matches the theoretical computational complexity of the algorithm.

Among all evaluated implementations, Java achieved the best overall performance, requiring approximately 241 ms to generate all subsets for an input size of 22 elements. The JVM efficiently optimized recursive function calls through Just-In-Time compilation while maintaining relatively low runtime overhead.

Go closely followed Java, completing the largest benchmark in approximately 295 ms. The Go runtime demonstrated efficient recursive execution and effective memory management, allowing excellent scalability despite the exponential growth of the recursion tree.

Rust produced highly competitive results with an average execution time of approximately 570 ms for the largest dataset. Its ownership model eliminated garbage collection overhead while maintaining deterministic memory management, resulting in predictable execution performance.

C++ demonstrated similar behavior to Rust, requiring approximately 592 ms for 22 elements. As a native compiled language, C++ efficiently executed recursive function calls, although the implementation exhibited slightly higher execution times than Java and Go.

C# completed the largest benchmark in approximately 675 ms. The Common Language Runtime effectively handled recursive execution, although automatic memory management introduced moderate runtime overhead compared with the fastest implementations.

Python exhibited the highest execution times by a substantial margin. The execution time increased from approximately 0.13 ms for five elements to nearly 70 seconds for twenty-two elements. The interpreted execution model, dynamic typing, recursive function overhead, and object allocation significantly amplified the computational cost of this exponential algorithm.

Overall, the benchmark clearly demonstrates how rapidly exponential algorithms become computationally expensive and highlights the importance of compiler optimization and runtime efficiency when executing deep recursive workloads.

9.7 Cross-Language Comparison

Subset generation places considerably greater emphasis on recursive function calls than any previous benchmark. Since every additional input element doubles the recursion tree, execution performance depends heavily on stack management, recursive call overhead, compiler optimization, and runtime implementation rather than arithmetic computation.

Java achieved the highest overall performance in this benchmark. The JVM effectively optimized recursive execution through adaptive Just-In-Time compilation, allowing Java to outperform the remaining implementations despite executing within a managed runtime environment.

Go also demonstrated exceptional scalability. Its lightweight runtime and efficient function call implementation enabled very competitive performance throughout the benchmark while maintaining relatively low memory overhead.

Rust and C++ produced very similar execution times. Both languages compile directly to native machine code and avoid interpreter overhead, resulting in efficient recursive execution. Rust additionally guarantees memory safety through its ownership model without sacrificing deterministic performance.

C# maintained stable execution behavior across all input sizes. Although garbage collection introduces additional runtime services, the CLR efficiently handled the recursive workload while remaining competitive with the native compiled implementations.

Python experienced the largest performance degradation because every recursive function call incurs interpreter overhead, dynamic type resolution, and object management. Since the number of recursive calls increases exponentially, these costs accumulate rapidly, producing execution times several orders of magnitude larger than those of the compiled languages.

9.8 Performance Ranking

Rank	Language	Observation
1	Java	Fastest recursive implementation with excellent JVM optimization.
2	Go	Outstanding scalability with efficient runtime recursion.

Rank	Language	Observation
3	Rust	High-performance native execution with guaranteed memory safety.
4	C++	Reliable native recursive implementation with consistent performance.
5	C#	Stable managed runtime with moderate recursion overhead.
6	Python	Slowest implementation because of interpreter and recursive function overhead.

9.9 Discussion

Subset generation represents the most computationally demanding benchmark included in this study because its execution time grows exponentially with the input size. Unlike polynomial-time algorithms such as sorting or matrix multiplication, compiler optimizations cannot reduce the inherent exponential complexity. Instead, improvements are primarily achieved by minimizing recursive function overhead and optimizing runtime execution.

The benchmark results indicate that Java handled recursive workloads most efficiently. Advanced Just-In-Time compilation enabled the JVM to optimize recursive execution and reduce function call overhead significantly during repeated executions.

Go also demonstrated remarkable performance. Its efficient runtime, optimized stack management, and lightweight execution model allowed the language to process increasingly large recursion trees while maintaining competitive execution times.

Rust and C++ produced comparable results, illustrating the effectiveness of native compilation for recursive algorithms. Rust additionally provides compile-time memory safety without introducing garbage collection overhead, making it an attractive choice for performance-critical recursive applications.

C# maintained predictable execution behavior despite operating within a managed runtime. Automatic memory management simplified implementation while introducing only moderate performance overhead.

Python experienced the greatest execution times because every recursive call passes through the interpreter and dynamic object system. As the recursion tree expands exponentially, interpreter overhead dominates the total execution time. Although Python remains an excellent language for education, prototyping, and algorithm development, it is less suitable for exhaustive recursive algorithms requiring maximum computational performance.

Overall, the benchmark emphasizes that runtime optimization and recursive execution efficiency become increasingly important as computational complexity grows exponentially.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
3. Sebesta, R. W. (2022). *Concepts of Programming Languages* (13th ed.). Pearson.
4. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
5. Klabnik, S., & Nichols, C. (2023). *The Rust Programming Language*. No Starch Press.
6. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
7. Skeet, J. (2019). *C# in Depth* (4th ed.). Manning Publications.
8. Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley.
9. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
10. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Source Code:

https://github.com/Caliber24/Algorithm-Benchmark.git